Vector Representations of Text in Classification Problems

Department of Mathematics, UAB

Academic Year 2024–2025

Author: Toni Esteve Gene

Lecturer: Dr. Antonio Lozano Bagén

Abstract

Vector representation of text is a fundamental component in machine learning, as it enables predictive models to process linguistic data efficiently in numerical form. In this context, the present work aims to formalize and mathematically analyze the process of vectorizing textual documents, focusing on three techniques: *TF-IDF* representation, latent semantic analysis (*LSA*), and latent Dirichlet allocation (*LDA*). The theoretical formalization covers the underlying mathematical foundations, as well as the properties and computational behavior of each method. Their mathematical bases are examined in detail: the construction of the classical vector space model through *TF-IDF*; the singular value decomposition (*SVD*) and the *Eckart–Young* theorem, applied to *LSA* for dimensionality reduction; and the generative probabilistic model underlying *LDA*, formalized through *Dirichlet* distributions and solved using inference techniques such as *Gibbs* sampling. In addition, algebraic and structural connections are explored through *Perron–Frobenius* theory, which justifies the emergence of latent semantic structures in document-term co-occurrence matrices.

Empirically, these representations have been applied to three heterogeneous textual datasets in order to compare their effectiveness in binary classification tasks. Supervised classification models (primarily a *Gradient Boosting* classifier) were trained to quantify the predictive capacity of each representation by means of the Area Under the ROC Curve (*AUC-ROC*). The results indicate that *TF-IDF* tends to provide the best overall predictive performance, followed by *LSA* and *LDA*. Nevertheless, both *LSA* and *LDA* provide competitive latent representations that are more compact and computationally more efficient, although the coherence of the representation depends on the semantic structure underlying the documents. Overall, this work highlights the importance of a solid mathematical foundation in text representation, showing that a rigorous formalization of these techniques contributes to improving both the interpretability and computational efficiency of text classification models.

Contents

1 Introduction **4**

2 Vector Representations of Documents and TF-IDF **5**

2.1 Dictionaries and Document Corpora 5

2.2 TF-IDF 5

2.3 Vector Representation of Documents 6

3 Latent Semantic Analysis **7**

3.1 Semantic Vector Space Model 7

3.2 Input Matrix 8

3.3 Decomposition of the Input Matrix into Orthogonal Components 9

3.3.1 Truncation of the Singular Value Decomposition 10

3.4 Semantic Connections and Latent Structure through Perron–Frobenius Theory 12

3.4.1 Introduction to Perturbations of Block-Diagonal Structure 16

3.5 Vector Representation of Documents and Similarities 18

4 Latent Dirichlet Allocation **18**

4.1 Joint Probability of the LDA Model 19

4.2 Computation of Topic Representations through Inference in LDA 20

4.3 Gibbs Sampling 21

4.3.1 Algorithm 21

4.3.2 Inference through Gibbs Sampling 22

4.3.3 Gibbs Sampling for Topic Assignment 22

4.4 Point Estimation of the Parameters θ_d and ϕ_k 26

4.4.1 Point Estimation of the Parameter θ_d 26

4.4.2 Point Estimation of the Parameter ϕ_k 28

4.5 Training an LDA Model 29

5 Methodology **30**

5.1 Spams dataset 30

5.2 Twitter dataset 31

5.3 20 Newsgroups Dataset 32

5.4 Summary of Results 33

6 Conclusions **34**

A Appendix **36**
A.1 Statistical Learning 36
A.1.1 Introduction 36
A.1.2 Supervised Learning 37
A.1.3 Classification and Regression 37
A.1.4 Loss Function 38
A.1.5 Metrics 39
A.1.6 The ROC Curve 41
A.2 Model Validation 42
A.2.1 K-Fold Cross Validation 42
A.2.2 Overfitting vs. underfitting 43
A.3 Introduction to Gradient Boosting Trees 43
A.3.1 Decision Tree 44
A.3.2 Boosting 45
A.3.3 Gradient Descent Method 45
A.4 Gradient Boosting Trees Method 46

B Code **48**

C Gibbs Sampling **48**
C.1 Introduction 48
C.2 Algorithm 48
C.3 Hammersley–Clifford Theorem and Positivity Condition (Discrete Case) . 49
C.4 Invariance of the Limiting Distribution (Discrete Case) 50
C.5 Irreducibility and Recurrence (Discrete Case) 50

1 Introduction

Within the fields of artificial intelligence and machine learning, data representation is a fundamental aspect that directly determines the efficiency and effectiveness of predictive models. In particular, when the input data take the form of text, their transformation into a suitable numerical representation constitutes both a conceptual and a computational challenge. This bachelor's thesis aims to study in depth the main techniques for vector representation of text, analyzing their mathematical foundations and evaluating their behavior in practical binary classification problems.

The motivation for this study is twofold. First, it arises from an interest in understanding, from a mathematical perspective, the mechanisms through which textual documents are projected into vector spaces in order to make it possible to apply statistical and machine learning methods to them. This issue is especially relevant given that all supervised machine learning algorithms ultimately rely on computational processes involving numerical data. It is therefore necessary to establish numerical representations of text that not only preserve the relevant information but also allow for computationally efficient processing.

Second, this work has been inspired by several courses taken throughout the *Bachelor's Degree in Mathematics*, such as *Statistics*, *Operations Research*, and *Linear Algebra*. The content of these courses has provided the conceptual tools required to rigorously address the study of techniques such as *singular value decomposition* and the *Gibbs sampling method*, which are key to the representations examined in this work. Likewise, the growing influence of artificial intelligence across multiple areas of society has highlighted the need to better formalize the theoretical foundations of these techniques, moving beyond the superficial treatment they are often given in the scientific literature.

The main contributions of this work are divided into two complementary lines. On the one hand, it presents a precise formalization of three of the most widely used techniques for transforming texts into vectors: *TF-IDF*, *latent semantic analysis (LSA)*, and *latent Dirichlet allocation (LDA)*. This formalization includes a description of their underlying mathematical foundations, their properties, and their computational behavior. Although the study builds on known results, it offers an original presentation that explores the mathematical development in greater depth, providing an academic perspective through careful formal derivations and a well-developed algebraic and probabilistic foundation. On the other hand, an empirical comparison designed by the author has been carried out on several textual datasets, using supervised learning models for binary classification. This analysis not only quantifies performance in terms of *AUC-ROC*, complexity, and computational cost, but also provides an original perspective by considering *LDA* as a vector representation technique useful in supervised contexts, rather than only as an unsupervised generative model.

Overall, this work seeks to contribute to the field of textual data representation by combining theoretical analysis with empirical validation, placing emphasis on formal clarity, mathematical motivation, and the transfer of this knowledge to concrete applications in text classification. Ultimately, it constitutes an exercise in synthesis between theory and practice, with the aim of understanding how mathematics can rigorously underpin the tools that make text processing possible.

2 Vector Representations of Documents and TF-IDF

With the aim of transforming textual documents into a numerical representation that can be efficiently processed by machine learning algorithms, it is essential to have a vector encoding of such documents. This transformation requires a prior structuring of the documents on the basis of a corpus and its corresponding dictionary, as well as the definition of a function that allows each document to be projected into a finite-dimensional space. Once these foundations have been established, this section introduces the representation known as *Term Frequency–Inverse Document Frequency (TF-IDF)*, one of the most widely used techniques in text processing due to its conceptual simplicity and ease of implementation.

2.1 Dictionaries and Document Corpora

Definició 1. *The document corpus is defined as the set of all text documents under analysis (Schütze, Manning, and Raghavan 2008, p. 19):*

$$D = \{d_1, d_2, \ldots, d_n\}$$

where each d_i represents a document in the corpus.

Definició 2. *Given a corpus of text documents $D = \{d_1, d_2, \ldots, d_n\}$, the dictionary is defined as the set of all unique terms appearing in the corpus Schütze, Manning, and Raghavan 2008, p. 22:*

$$\mathcal{V} = \bigcup_{d \in D} T_d$$

where T_d denotes the set of terms appearing in document d.

Given a corpus of text documents $D = \{d_1, d_2, \ldots, d_n\}$, the dictionary $\mathcal{V}$ is understood as a finite set of terms typically selected according to a frequency criterion (in most cases, removing words that do not provide relevant information for the analysis, such as articles, pronouns, and so forth, commonly referred to as *stopwords*). Formally, a threshold $k \in \mathbb{N}$ is established, representing the maximum number of terms to be included in the dictionary (which usually becomes a hyperparameter to be tuned when applying a supervised learning model [A.1.2]):

$$\mathcal{V}_k = \{t_1, t_2, \ldots, t_k\}, \quad \text{where } t_i \text{ is one of the } k \text{ most frequent terms in } D.$$

2.2 TF-IDF

TF-IDF (Term Frequency–Inverse Document Frequency) is a function that assigns a weight to each term in a document according to its frequency within that document and its rarity across a corpus of documents.

Definició 3. *Let $D = \{d_1, d_2, \ldots, d_n\}$ be a corpus of documents and let $\mathcal{V}$ be the vocabulary of terms appearing in D. We define the term frequency (TF) of a term $t \in \mathcal{V}$ in a document $d \in D$ as follows (Schütze, Manning, and Raghavan 2008, p. 117):*

$$tf(t, d) = \frac{f(t, d)}{\max\{f(t', d) \mid t' \in d\}}$$

where:

- $f(t, d)$ *is the number of times that term* t *appears in document* d.
- $\max\{f(t', d) \mid t' \in d\}$ *is the maximum frequency of any term in document* d.
- ***Purpose:*** *To normalize the term frequency with respect to the most frequent words in the document, thereby avoiding biases in longer texts.*

Definició 4. *Inverse document frequency (IDF) is defined as follows (Schütze, Manning, and Raghavan 2008, p. 118):*

$$idf(t, D) = \log\left(\frac{|D|}{|\{d \in D \mid t \in d\}|}\right)$$

where:

- $|D|$ *is the total number of documents in the corpus.*
- $|\{d \in D \mid t \in d\}|$ *is the number of documents in which term* t *appears.*
- ***Purpose:*** *To penalize terms that appear in many documents, since they contribute less discriminative information.*

Definició 5. *The TF-IDF weight of a term* t *in a document* d *is defined as follows (Schütze, Manning, and Raghavan 2008, p. 119):*

$$tfidf(t, d) = tf(t, d) \times idf(t, D)$$

- *This value combines the local importance of the term within a document (TF) with its global importance across the corpus (IDF).*
- *Terms that are frequent in a document but rare in the corpus receive a high TF-IDF value, making them more relevant for text classification and analysis.*

2.3 Vector Representation of Documents

In a *TF-IDF* problem, the documents in the corpus $D = \{d_1, d_2, \ldots, d_n\}$ are represented in a vector space of dimension $|\mathcal{V}|$, where $\mathcal{V}$ is the dictionary of terms.

The vector representation of a document d_i is defined as:

$$\mathbf{v}_{d_i} = (tfidf(t_1, d_i), tfidf(t_2, d_i), \ldots, tfidf(t_{|\mathcal{V}|}, d_i)) \in \mathbb{R}^{|\mathcal{V}|}$$

where:

- Each component of the vector $\mathbf{v}_{d_i}$ corresponds to the *TF-IDF* weight of term $t_j \in \mathcal{V}$ in document d_i.
- The length of the vector is $|\mathcal{V}|$, which is the total number of terms in the dictionary.

This numerical representation of documents facilitates the application of machine learning models.

3 Latent Semantic Analysis

Latent Semantic Analysis (*LSA*) is a technique used for information retrieval in text whose objective is to describe the latent relationships between words and documents. This method seeks to represent the associations that exist among different terms according to their joint occurrence in different contexts. These representations suggest the implicit presence of thematic clusters and semantic networks, through which it becomes possible to organize words according to their conceptual and thematic similarity.

In the following sections, the theoretical foundations necessary to understand the semantic vector space model used in *LSA* will be developed (Leydesdorff and Welbers 2011; Deerwester et al. 1990; Landauer and Dumais 1997). First, the definition and structure of a semantic vector space will be introduced formally, specifying how the document corpus and the vocabulary are represented within this space. Next, the construction of the input matrix will be presented, containing the weights of terms in relation to the documents of a specific corpus. Finally, the decomposition of this matrix by means of singular value decomposition (*SVD*) and *Perron–Frobenius theory* will be analyzed in detail, exploring the relevance of this decomposition for reducing dimensionality and obtaining an efficient and meaningful representation of the latent semantic content of texts.

3.1 Semantic Vector Space Model

Definició 6. *Let $\mathcal{D} = \{d_1, d_2, \ldots, d_n\}$ be a corpus of documents, and let $\mathcal{V}$ be a vocabulary of size m. We define a semantic vector space as a pair $(\mathbb{R}^m, \phi)$, where $\mathbb{R}^m$ is a Euclidean vector space of dimension m, in which each document is represented as a vector in that space, and $\phi : \mathcal{D} \to \mathbb{R}^m$ is an assignment function that maps each document to a vector in $\mathbb{R}^m$.*

Each element $d_i \in \mathcal{D}$ is represented by a vector $\mathbf{v}_i \in \mathbb{R}^m$, where the structure of this vector depends on the representation method employed (Berry and Browne 2005, p. 4). One of the most popular representations is the one based on *TF-IDF*, as explained above.

This structure makes it possible to perform algebraic operations in order to measure similarities and extract latent information within a large corpus of documents D. Moreover, through this representation, one may assume that the meaning of documents can be derived directly from the vocabulary $\mathcal{V}$ from which they are composed.

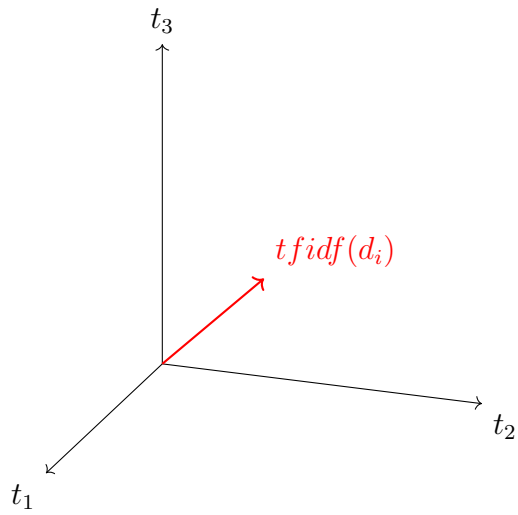

Figure 1: Vector representation of a document in a vocabulary consisting of three terms. *Source: Author's own elaboration.*

3.2 Input Matrix

Considering the previous definition of a semantic vector space model, let us assume that we have a document corpus $\mathcal{D}$ with $|\mathcal{D}| = n$ documents and an associated vocabulary $\mathcal{V}$ with $|\mathcal{V}| = m$ terms. From this representation, we organize the documents into an input matrix $A \in \mathbb{R}^{m \times n}$, where each column corresponds to the vector $\mathbf{v}_j = \phi(d_j)$ representing the document $d_j \in \mathcal{D}$ within the semantic vector space previously defined through the assignment function ϕ. The entries a_{ij} of this matrix are determined by weighting functions such as *TF-IDF*, thus reflecting the relevance of term t_i in document d_j. This matrix representation therefore captures both the global importance of terms across the entire collection of documents and their local specificity within each individual document (Leydesdorff and Welbers 2011, p. 3).

$$
\begin{array}{c}
 \\ t_1 \\ t_2 \\ \vdots \\ t_i \\ \vdots \\ t_m
\end{array}
\begin{array}{c}
\begin{array}{cccccc} d_1 & d_2 & \dots & d_j & \dots & d_n \end{array} \\
\begin{bmatrix}
a_{11} & a_{12} & \cdots & a_{1j} & \cdots & a_{1n} \\
a_{21} & a_{22} & \cdots & a_{2j} & \cdots & a_{2n} \\
\vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\
a_{i1} & a_{i2} & \cdots & a_{ij} & \cdots & a_{in} \\
\vdots & \vdots & \ddots & \vdots & \ddots & \vdots \\
a_{m1} & a_{m2} & \cdots & a_{mj} & \cdots & a_{mn}
\end{bmatrix}
\end{array}
$$

Figure 2: Schematic representation of the input matrix A, where the rows correspond to terms t_i and the columns correspond to documents d_j. *Source: Author's own elaboration.*

In general, the input matrix A tends to exhibit a sparse structure, since each document usually employs only a reduced subset of the terms available in the vocabulary. This is

a consequence of the fact that documents often address specific topics, which naturally limits the number of terms used in each of them.

3.3 Decomposition of the Input Matrix into Orthogonal Components

From an algebraic point of view, the matrix A defined above contains information about how terms are associated with documents and how documents share terms. This double intersection of information implies that the relationships between terms and documents are not independent, but rather correlated. In other words, the occurrence of a given term in a document is often conditioned by the presence of other terms, and documents can be grouped according to common semantic patterns.

These interdependencies are manifested algebraically in the fact that the matrix A exhibits linear or nearly linear dependencies among rows and/or columns (for example, given the vector representation of the word *beach* across documents, it is highly likely to be very similar to the vector representation of the word *sea*) and a correlation structure visible in the matrices A^TA and AA^T, which indicate, respectively, similarities between documents and between terms.

This situation suggests the existence of an underlying latent structure, that is, a reduced set of latent semantic factors that explain term co-occurrence and document similarity. However, this structure is not captured in the original representation of A, since documents and terms are expressed in canonical bases that do not explicitly reflect these deeper conceptual relationships (Landauer and Dumais 1997, p. 214).

It is precisely in this context that singular value decomposition (SVD) becomes a fundamental tool. Let us recall the statement given in (Dongarra et al. 2018, p. 810):

Teorema 1. *(Singular Value Decomposition) Let $A \in \mathbb{R}^{m\times n}$ be a real matrix of dimension $m\times n$. Then there exist two orthogonal matrices $U \in \mathbb{R}^{m\times m}$ and $V \in \mathbb{R}^{n\times n}$, and a diagonal matrix $\Sigma \in \mathbb{R}^{m\times n}$ such that:*

$$A = U\Sigma V^T$$

where:

- *U is an orthogonal matrix, that is, $U^TU = I_m$.*
- *V is an orthogonal matrix, that is, $V^TV = I_n$.*
- *Σ is a diagonal matrix of the form:*

$$\Sigma = \begin{bmatrix} \sigma_1 & 0 & \cdots & 0 & 0 & \cdots & 0 \\ 0 & \sigma_2 & \cdots & 0 & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sigma_r & 0 & \cdots & 0 \\ 0 & 0 & \cdots & 0 & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 0 & 0 & \cdots & 0 \end{bmatrix} \in \mathbb{R}^{m \times n}$$

where $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0$ *are the singular values of* A*, and* $r = \text{rank}(A)$*.*

- *The first* r *columns of* U *are the left singular vectors, that is, the eigenvectors of* AA^T*.*
- *The first* r *columns of* V *are the right singular vectors, that is, the eigenvectors of* A^TA*.*
- *The singular values* σ_i *are the non-negative square roots of the non-zero eigenvalues of* A^TA *(or equivalently of* AA^T*).*

The *SVD* $A = U\Sigma V^T$ provides an orthogonal decomposition that separates and restructures the latent relationships between terms and documents through three transformations. On the one hand, V identifies orthogonal combinations of documents that represent independent semantic dimensions in the input space; Σ quantifies the relevance of each of these dimensions through the singular values σ_i; and U expresses how these dimensions are manifested in the term space, defining a new orthogonal basis for the output space.

This decomposition shows that $Av_i = \sigma_i u_i$, that is, each orthogonal combination of documents v_i is transformed by A into an orthogonal combination of terms u_i, scaled by σ_i. In this way, *SVD* diagonalizes the relationship between documents and terms in a latent space, providing a decoupled and structured representation that reveals the essential components of the semantic content of the corpus (Deerwester et al. 1990, p. 398).

3.3.1 Truncation of the Singular Value Decomposition

In high-dimensional and sparse settings, as is typically the case in document analysis, *SVD* makes it possible to identify a set of latent dimensions that capture underlying semantic patterns between terms and documents. However, not all of these dimensions contribute significantly to the representation of the content: as will be seen, the dimensions associated with the smallest singular values often capture less relevant variations or statistical noise. For this reason, a common practice in such cases is to apply a truncated *SVD*, consisting of removing the latent dimensions of lower weight and retaining only the first k most informative components. This reduction not only simplifies the model and improves its interpretability, but also preserves most of the relevant semantic variability of the corpus, making it possible to uncover hidden semantic connections by grouping terms and documents with similar meanings into shared conceptual dimensions, thereby facilitating synonym identification and improving information retrieval without requiring exact word matches (Martin and Berry 2007, p. 42).

From the *SVD*, we may write $A \in \mathbb{R}^{m \times n}$, where $r = \text{rank}(A)$, as

$$A = U_r \Sigma_r V_r^T$$

where $U_r = [u_1, u_2, ..., u_r] \in \mathbb{R}^{m \times r}$ is a matrix containing the first r columns of U (which are orthogonal to one another), $V_r = [v_1, v_2, ..., v_r] \in \mathbb{R}^{n \times r}$ is a matrix containing the first r columns of V (which are orthogonal to one another), and $\Sigma_r \in \mathbb{R}^{r \times r}$ is a diagonal matrix containing the nonzero singular values of A, $\sigma_1 \geq \sigma_2 \geq ... \geq \sigma_r > 0$.

Thus, if we reduce the dimensionality of the matrix A by selecting only the first k most significant terms ($k < r$), we eliminate the concepts associated with the smaller singular values, which are less important in the representation of the model, obtaining the matrix

$$A_k = U_k \Sigma_k V_k^T$$

where $U_k = [u_1, u_2, ..., u_k] \in \mathbb{R}^{m \times k}$ is a matrix containing the first k columns of U (which are orthogonal to one another), $V_k = [v_1, v_2, ..., v_k] \in \mathbb{R}^{n \times k}$ is a matrix containing the first k columns of V (which are orthogonal to one another), and $\Sigma_k \in \mathbb{R}^{k \times k}$ is a diagonal matrix containing the k largest singular values of A, $\sigma_1 \geq \sigma_2 \geq ... \geq \sigma_k > 0$ (Horasan et al. 2019, p. 2).

The theoretical foundation of this approach is provided by the *Eckart–Young–Mirsky Theorem*, which guarantees that the matrix A_k is the best rank-k approximation of the original matrix A (minimizing either the 2-norm or the Frobenius norm [1])

Teorema 2. *(Eckart and Young 1936, p. 211) Let*

$$D = U \Sigma V^\top \in \mathbb{R}^{m \times n}, \quad m \geq n$$

be the singular value decomposition (SVD) of D, where $\Sigma := \text{diag}(\sigma_1, \ldots, \sigma_m)$ is the rectangular diagonal matrix of size $m \times m$ with singular values $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_m$.

For $r \in \{1, \ldots, m-1\}$, partition U, Σ, and V as follows:

$$U = \begin{bmatrix} U_1 & U_2 \end{bmatrix}, \quad \Sigma = \begin{bmatrix} \Sigma_1 & 0 \\ 0 & \Sigma_2 \end{bmatrix}, \quad V = \begin{bmatrix} V_1 & V_2 \end{bmatrix},$$

where U_1 has size $m \times r$, Σ_1 has size $r \times r$, and V_1 has size $n \times r$.

Then, the rank-r matrix obtained from the truncated singular value decomposition (truncated SVD) is defined by

$$\widehat{D}^* = U_1 \Sigma_1 V_1^\top,$$

such that

[1] The Frobenius norm of A, denoted by $\|A\|_F$, is defined as

$$\|A\|_F = \sqrt{\sum_{i=1}^{m} \sum_{j=1}^{n} |a_{ij}|^2}.$$

$$\|D - \widehat{D}^*\|_F = \min_{\operatorname{rank}(\widehat{D}) \leq r} \|D - \widehat{D}\|_F = \sqrt{\sigma_{r+1}^2 + \cdots + \sigma_m^2}.$$

The minimizer $\widehat{D}^*$ *is unique if and only if* $\sigma_{r+1} \neq \sigma_r$. (See the proof in (Eckart and Young 1936, p. 216))

The theorem not only states that A_k is the solution to the problem

$$\min_{\operatorname{rank}(B) \leq k} \|A - B\|_F,$$

but in particular also yields

$$\|A - A_k\|_F = \sqrt{\sigma_{k+1}^2 + \sigma_{k+2}^2 + \cdots + \sigma_r^2}.$$

In other words, the singular values σ_i determine the *amount of information* in each direction, and by truncating the *SVD* to the k most important dimensions, the contribution of the directions associated with $\sigma_{k+1}, \sigma_{k+2}, \ldots, \sigma_r$ is eliminated, which is precisely what is lost in the best rank-k approximation.

3.4 Semantic Connections and Latent Structure through Perron–Frobenius Theory

From a conceptual point of view, it has been observed that *LSA* makes it possible to identify latent relationships between documents and terms through their grouping into latent concepts, which capture the maximum variance of the information contained in the original matrix A, as stated by the *Eckart–Young Theorem* [2]. Nevertheless, it is natural to ask why this method is capable of revealing semantic relationships between documents that do not explicitly share any common term, yet still address thematically similar content. Likewise, it is necessary to understand how synonymous terms that do not co-occur in any document of the corpus can nevertheless end up being represented jointly within the same latent structure.

This fact can be rigorously justified through the application of *Perron–Frobenius Theory*, specifically in the context of the analysis of nonnegative matrices, considering the distinction between reducible and irreducible matrices. This theory provides the structural conditions under which the semantic relationships among documents (or terms) can be decomposed into disjoint blocks (in the reducible case), or instead exhibit a strongly connected structure (in the irreducible case), which has direct implications for the ability of *LSA* to capture global semantic relationships through singular value decomposition (Valle-Lisboa and Mizraji 2007, p. 15).

Let $A \in \mathbb{R}^{m \times n}$ be the term-document matrix, where each column represents a document $d_j \in \mathbb{R}^m$ and each row represents a term in the vocabulary V. Define the matrix $D = A^\top A \in \mathbb{R}^{n \times n}$, which measures similarity between documents in the term space. Each entry D_{ij} of the matrix D is the inner product between the vectors d_i and d_j, that is,

$$D_{ij} = \langle d_i, d_j \rangle = \sum_{k=1}^{m} A_{k,i} A_{k,j},$$

which measures the degree of co-occurrence of the terms in the vocabulary $\mathcal{V}$ between documents d_i and d_j. Thus, the matrix D acts as a document similarity matrix based on the information distributed across terms, which we shall call the *co-occurrence matrix*.

Thus, if we consider the singular value decomposition of the matrix A, namely $A = U\Sigma V^\top$, we obtain the diagonalization of the matrix D

$$D = A^\top A = (U\Sigma V^\top)^\top (U\Sigma V^\top) = V\Sigma^\top U^\top U\Sigma V^\top = V\Sigma^2 V^T \tag{1}$$

We now introduce key concepts (Meyer 2000, p. 674) from *Perron–Frobenius Theory*, which establishes special properties of the eigenvalues and eigenvectors of nonnegative reducible and irreducible matrices.

Definició 7. *(Reducible and Irreducible Matrix). Let $D \in \mathbb{R}^{n\times n}$ be a square matrix. The matrix D is said to be reducible if there exists a permutation matrix $P \in \mathbb{R}^{n\times n}$ such that:*

$$P^T DP = \begin{pmatrix} X & Y \\ 0 & Z \end{pmatrix},$$

where $X \in \mathbb{R}^{k\times k}$ and $Z \in \mathbb{R}^{(n-k)\times(n-k)}$ are square matrices for some $1 \leq k < n$, and 0 denotes the zero matrix of dimension $(n-k)\times k$. Otherwise, D is said to be irreducible.

Definició 8. *(Block-Decomposable Matrix). A matrix $D \in \mathbb{R}^{n\times n}$ is said to be decomposable (or block-diagonal) if there exists a permutation matrix $P \in \mathbb{R}^{n\times n}$ such that D can be expressed as a block-diagonal matrix, that is, a matrix of the form:*

$$P^T DP = \begin{pmatrix} D_1 & 0 & \cdots & 0 \\ 0 & D_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & D_m \end{pmatrix},$$

where each D_i is a square matrix, and the off-diagonal blocks are zero.

Definició 9. *Let $A = (a_{ij}) \in \mathbb{R}^{n\times n}$ be a matrix. We define the directed graph associated with A, denoted by $\mathcal{G}(A)$, as the graph with node set $\{N_1, N_2, \ldots, N_n\}$ in which there exists a directed edge from N_i to N_j if and only if $a_{ij} \neq 0$.*

Definició 10. *The graph $\mathcal{G}(A)$ is said to be strongly connected if, for every pair of nodes (N_i, N_k), there exists a directed path from N_i to N_k; that is, there exists a sequence of directed edges connecting N_i to N_k.*

In fact, the following characterization can be given:

Lema 1. *A matrix $A \in \mathbb{R}^{n\times n}$ is irreducible if and only if the graph $\mathcal{G}(A)$ is strongly connected (see the proof in (Meyer 2000, p. 742)).*

Teorema 3. *(Perron–Frobenius Theorem for Irreducible Nonnegative Matrices). Let $D \in \mathbb{R}^{n\times n}$ be an irreducible and nonnegative matrix, that is, $D \geq 0$. Then the following properties hold:*

- *The spectral radius $r = \rho(D)$, defined as the maximum absolute value among the eigenvalues of D, is real, positive, and has algebraic multiplicity equal to 1.*

- *There exists an eigenvector* $\mathbf{v} \in \mathbb{R}^n$ *such that* $\mathbf{v} > 0$ *and* $D\mathbf{v} = r\mathbf{v}$.
- *There exists a left eigenvector* $\mathbf{p} \in \mathbb{R}^n$, *with* $\mathbf{p} > 0$ *and* $\|\mathbf{p}\|_1 = 1$, *such that* $\mathbf{p}^\top D = r\mathbf{p}^\top$. *This vector* $\mathbf{p}$ *is called the Perron vector, and the pair* $(r, \mathbf{p})$ *is called the Perron pair. No other nonnegative eigenvector of* D *is admissible, except for positive multiples of* $\mathbf{p}$, *regardless of the associated eigenvalue. (See the proof in (Khim 2007, p. 2))*

Let $D = A^\top A$ be the document co-occurrence matrix obtained from the term-document matrix A. By construction, D is a symmetric and nonnegative matrix, since each entry D_{ij} corresponds to the inner product between the vectors representing documents d_i and d_j.

Since D is symmetric, by the *Spectral Theorem* [2] it can be diagonalized by an orthonormal basis, and all its eigenvalues are real (as seen in (1)). This property facilitates the application of the *Perron–Frobenius Theorem* [3], which studies the spectral behavior of nonnegative matrices.

Let us now interpret D as the adjacency matrix of a directed graph $G(D)$, where each node represents a document and there exists a directed edge between the nodes $d_i \to d_j$ if $D_{ij} \neq 0$, that is, if the documents share at least one term. In this context, the irreducibility of D is equivalent to the graph being strongly connected [10]; this means that, for any pair of documents (d_i, d_j), there exists a path (potentially passing through intermediate terms) connecting d_i to d_j, and vice versa (Valle-Lisboa and Mizraji 2007, p. 17).

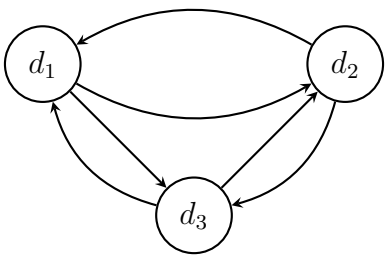

Figure 3: Example of a strongly connected directed graph associated with an adjacency matrix D, where each node represents a document and the edges indicate the existence of shared terms. The fact that the graph is strongly connected guarantees the irreducibility of D, ensuring that there exists a directed path between any pair of documents. *Source: Author's own elaboration.*

Applying the *Perron–Frobenius Theorem* [3] to the matrix D, it follows that if D is irreducible, then its dominant eigenvalue $\rho(D)$ is strictly positive and there exists an associated strictly positive eigenvector $\mathbf{v}_1 > 0$. In this context, $\mathbf{v}_1$ reflects the presence

[2] *Spectral Theorem* (version for real symmetric matrices): Let $A \in \mathbb{R}^{n\times n}$ be a symmetric matrix. Then there exists an orthonormal basis of eigenvectors of A. That is, there exists an orthogonal matrix $Q \in \mathbb{R}^{n\times n}$ (where $Q^\top Q = I$) such that:

$$A = Q\Lambda Q^\top,$$

where Λ is a diagonal matrix containing the eigenvalues of A, all of which are real.

of a global latent structure that semantically connects all documents, even if they do not explicitly share the same terms.

The contrapositive of this result implies that if the dominant eigenvector contains zero coordinates, this is indicative that D is reducible, and therefore there exist subsets of documents that are semantically disconnected from the others within the latent space (they do not share any term with the rest, or there is no chain of co-occurrence connecting them).

In general, it is not true that the existence of an eigenvector with strictly positive coordinates for a nonnegative matrix implies that the matrix is irreducible. However, in the specific case in which the matrix is symmetric, this statement does hold. As shown by the following lemma, the fact that the dominant eigenvector of D is strictly positive guarantees that the associated co-occurrence graph is strongly connected; in our context, this means that all documents are connected through some semantic path within the latent structure represented by D.

Lema 2. *Let $D = A^\top A$ be a symmetric and nonnegative matrix of dimension $n \times n$. Let σ_1 be the dominant eigenvalue of D (that is, the one with maximum modulus). Suppose that σ_1 has algebraic multiplicity 1 and that there exists an eigenvector $\mathbf{v}_1 \in \mathbb{R}^n$ associated with σ_1 such that $\mathbf{v}_1 > 0$ (all of its components are strictly positive). Then, D is irreducible, that is, the associated directed graph $\mathcal{G}(D)$ is strongly connected. (See the proof in (Valle-Lisboa and Mizraji 2007, p. 18))*

Although in some cases the co-occurrence matrix $D = A^\top A$ is irreducible and, therefore, all documents are semantically connected through a co-occurrence path, in practice it may happen that D is reducible. This situation arises when the collection of documents contains completely disconnected thematic subsets, that is, groups of documents that do not share any term in common with the rest. From an algebraic point of view, the reducibility of D is manifested through the appearance of strict zeros in the first singular vector of A, indicating the presence of orthogonal components within the latent space. In this scenario, the following property allows this structure to be interpreted as a decomposition into semantically independent blocks, each of which can be analyzed separately.

Lema 3. *Let $D_{n\times n}$ be a symmetric matrix. If D is reducible, then it is decomposable (that is, it can be written as a block-diagonal matrix).*

For our matrix $D = A^T A$, this indicates that the corpus contains mutually unrelated subsets of documents: documents that do not share any term with other groups. Each resulting diagonal block corresponds to an independent thematic cluster, representing a coherent grouping of documents dealing with the same topic, yet semantically disconnected from the other clusters. This observation implies that the directed graph $\mathcal{G}(D)$ associated with the co-occurrence matrix D contains several strongly connected components, each of which corresponds to an autonomous thematic block within the corpus. This reflects the fact that the documents within each component are semantically interconnected, whereas no co-occurrence path exists connecting them to documents in other components (Valle-Lisboa and Mizraji 2007, p. 20).

3.4.1 Introduction to Perturbations of Block-Diagonal Structure

Although the analysis of the matrix $A^\top A$ as a decomposable matrix is theoretically clear when documents are grouped into completely separate thematic blocks, in practice this situation is uncommon. Documents belonging to different topics may often share occasional or marginally relevant terms, thereby generating weak correlations between apparently independent blocks. These correlations induce a structure which, although close to block-diagonal, incorporates small off-diagonal perturbations that connect the different blocks, albeit only weakly (Valle-Lisboa and Mizraji 2007, p. 22).

This phenomenon can be modeled by considering a document-document correlation matrix of the form:

$$A^\top A = M = D + \varepsilon E,$$

where D represents a block-diagonal (decomposable) matrix reflecting the ideal thematic structure, and εE is a symmetric perturbation that captures spurious correlations between documents belonging to different blocks. Under this formulation, M preserves symmetry and non-negativity, but introduces additional spectral complexity that may affect the interpretation of the singular vectors.

To illustrate the complexity of this new case and study its effect on eigenvalues and eigenvectors, we analyze a concrete example with two weakly connected topics. This example (Valle-Lisboa and Mizraji 2007, p. 22) shows how small perturbations may alter the behavior of the eigenvectors and, therefore, the detection of topics through singular value decomposition.

Consider the matrices $D \in \mathbb{R}^{2\times 2}$ and $E \in \mathbb{R}^{2\times 2}$ defined by:

$$D = \begin{pmatrix} a & 0 \\ 0 & b \end{pmatrix}, \quad E = \begin{pmatrix} d & z \\ z & f \end{pmatrix}$$

where $a, b, d, f, z \in \mathbb{R}$ and $\varepsilon > 0$ is a small parameter controlling the magnitude of the perturbation.

Define the perturbed matrix:

$$M = D + \varepsilon E = \begin{pmatrix} a + \varepsilon d & \varepsilon z \\ \varepsilon z & b + \varepsilon f \end{pmatrix}$$

This matrix is symmetric by construction, and the eigenvalues λ_1, λ_2 of M can be computed exactly as:

$$\lambda_{1,2} = \frac{(a + \varepsilon d) + (b + \varepsilon f)}{2} \pm \frac{1}{2}\sqrt{[(a + \varepsilon d) - (b + \varepsilon f)]^2 + 4\varepsilon^2 z^2}$$

For simplicity, define $\Lambda := a + \varepsilon d$ and $\eta := b + \varepsilon f$, yielding:

$$\lambda_{1,2} = \frac{\Lambda + \eta}{2} \pm \frac{1}{2}\sqrt{(\Lambda - \eta)^2 + 4\varepsilon^2 z^2}$$

This expression shows how the perturbation εz modifies the spectrum of D, which was originally $\{a, b\}$. The eigenvectors associated with the eigenvalues λ_1, λ_2 are

$$\mathbf{v}_1 = \frac{1}{N}\begin{pmatrix} 1 \\ \frac{\varepsilon z}{L} \end{pmatrix}, \quad \mathbf{v}_2 = \frac{1}{N}\begin{pmatrix} -\frac{\varepsilon z}{L} \\ 1 \end{pmatrix}$$

where

- $L = \frac{1}{2}\left(|\Lambda - \eta| + \sqrt{(\Lambda - \eta)^2 + 4\varepsilon^2 z^2}\right)$
- N is the normalization factor: $N = \sqrt{1 + \left(\frac{\varepsilon z}{L}\right)^2}$

At first glance, one observes that when $\varepsilon = 0$, the ideal decomposable situation is recovered, in which the eigenvectors are

$$\mathbf{v}_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad \mathbf{v}_2 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

This implies that there is no correlation between the two documents (completely separate blocks).

For $\varepsilon > 0$, off-diagonal components appear, introducing correlation between the two documents. As a result, the first eigenvector $\mathbf{v}_1$ will have strictly positive components if $\varepsilon z > 0$, which suggests a semantic connection between the documents, as stated by the *Perron–Frobenius Theorem* [3], and this connection will depend on the magnitude of the perturbation.

The second eigenvector $\mathbf{v}_2$, corresponding to the second largest eigenvalue of the co-occurrence matrix $D = A^\top A$, defines a direction orthogonal to the principal direction $\mathbf{v}_1$. This direction can be interpreted as a latent axis of semantic variation, in which documents with components of opposite sign in $\mathbf{v}_2$ are projected onto differentiated regions of the latent space. This suggests that $\mathbf{v}_2$ discriminates between two groups of documents that may correspond to different topics or semantic structures within the corpus.

In conclusion, the example analyzed shows how small off-diagonal perturbations may break the ideal block-diagonal structure and introduce weak semantic connections between apparently disconnected documents. These interactions are directly reflected in the eigenvectors of the co-occurrence matrix, which move from having non-overlapping components to incorporating mixed coordinates that capture latent correlations. The spectral analysis of this situation, grounded in *Perron–Frobenius theory*, provides a theoretical framework for understanding how *LSA* detects the presence of thematic blocks shared among documents and how, despite the perturbations, a certain capacity to discriminate differentiated semantic structures in the latent space is preserved.

Analogously, it is also possible to carry out an equivalent study on the matrix $W = AA^\top \in \mathbb{R}^{m\times m}$, which encodes similarity between terms on the basis of the documents in which they appear. In this case, W may be interpreted as a term co-occurrence matrix, and its eigenvectors correspond to the left singular vectors of the matrix A. The singular value decomposition $A = U\Sigma V^\top$ guarantees that $W = AA^\top = U\Sigma^2 U^\top$ is symmetric and diagonalizable, and its spectral structure allows latent groupings of terms to be analyzed in a manner completely symmetric to the document case. Thus, the first eigenvector u_1 of W may also be regarded as a vector of global semantic correlation among terms, and its structure may be analyzed using the same principles of *Perron–Frobenius theory.* This approach provides a complementary perspective to the analysis of D, making it possible to identify networks of meaning among words that do not explicitly co-occur, but are connected through a path of documents via intermediate terms.

3.5 Vector Representation of Documents and Similarities

One of the main advantages of latent semantic analysis lies in the fact that it not only allows documents to be represented as vectors in a reduced-dimensional space, but also facilitates the computation of similarity measures between terms or between documents themselves, without the need to operate directly on the original high-dimensional matrix. By means of projection onto the latent space generated by truncating the singular value decomposition, comparison operations can be carried out efficiently and accurately in a low-dimensional orthonormal subspace, preserving the relevant semantic information and significantly reducing computational cost. Let $d_i \in \mathbb{R}^m$ be the vector representation of a document, obtained from the *TF-IDF* weights, and let $e_i \in \mathbb{R}^n$ be the canonical vector associated with the i-th column, so that $d_i = Ae_i$. Considering the truncated decomposition $A_k = U_k \Sigma_k V_k^T$, one obtains the approximation:

$$d_i = Ae_i \approx A_k e_i = U_k \Sigma_k v_i$$

where $v_i \in \mathbb{R}^k$ is the i-th row of V_k^T, expressing the projection of the document d_i onto the first k principal latent components. Thus, if only the representation of the document in the latent space of dimension k is retained, one obtains the vector (Martin and Berry 2007, p. 49):

$$\hat{d}_i = v_i = \Sigma_k^{-1} U_k^T d_i \in \mathbb{R}^k$$

which constitutes the reduced and semantically meaningful version of d_i in terms of the k latent concepts identified by the truncated *SVD*. The same expression applies more generally to project any document, whether belonging to the training set (as in the present argument) or to the test set, into the reduced semantic space, thereby guaranteeing a coherent and comparable representation within the constructed latent model.

4 Latent Dirichlet Allocation

Latent Dirichlet Allocation (*LDA*) is a probabilistic generative model for a corpus. A generative process, in the context of probabilistic models, is a mathematical description of how the observed data (in *LDA*, the documents in a corpus) may have been generated from underlying latent variables and probability distributions. These generative models define a sequence of probabilistic steps that model the creation of the data, making it possible to generate new synthetic samples following the same probabilistic structure (Jelodar et al. 2019, p. 5).

The generative process in *LDA* describes how a document in a corpus is generated, based on the assumption that each document is composed of a mixture of latent topics and that each word in the document arises from one of these topics (Blei, Ng, and Jordan 2003, p. 996). Let us define a dictionary $\mathcal{V}$ from our corpus of documents D, together with a fixed number of latent topics K within the corpus. Then, for each latent topic $k = 1, \ldots, K$, a distribution over all the words in the vocabulary is assigned, so that

$$\phi^{(k)} = (\phi_1^{(k)}, \phi_2^{(k)}, ..., \phi_{\mathcal{V}}^{(k)}) \sim Dirichlet(\beta)^3$$

[3]The *Dirichlet* distribution is a probability distribution defined over probability vectors, that is, over collections of non-negative numbers that sum to 1. It is parameterized by a vector of positive parameters

The steps of the generative process in *LDA* for each document $\mathbf{w}$ in a corpus D are as follows:

1. Determine the length of the document: the number of words N in the document is chosen according to a Poisson distribution:

$$N \sim Poisson(\zeta)$$

2. Choose the topic distribution for the document: a topic distribution θ for the document is drawn from a *Dirichlet* distribution with parameter α:

$$\theta \sim Dirichlet(\alpha)$$

   This defines how the topics are distributed within the document.

3. Generate words for the document: for each word w_n in the document:

   - A topic z_n for the word is chosen from a multinomial distribution based on θ:

$$z_n \sim Multinomial(\theta)$$

     Next, a word w_n is chosen from a multinomial distribution depending on the topic z_n:

$$w_n \sim Multinomial(\phi^{(z_n)})$$

     The random vector $\phi^{(z_n)}$ stores the probability with which each word appears within topic z_n.

4.1 Joint Probability of the LDA Model

The chain rule of probability establishes how to compute the joint probability of a set of random variables from their conditional probabilities.

Lema 4. *(Chain Rule of Probability) For a set of n random variables $X_1, X_2, \ldots, X_n$, the joint probability can be decomposed as follows:*

$$p(X_1, X_2, \ldots, X_n) = p(X_1)p(X_2 \mid X_1)p(X_3 \mid X_1, X_2) \ldots p(X_n \mid X_1, \ldots, X_{n-1})$$

(See the proof in (Ferrante and Bardina 2023, p. 6))

Given a document d with N words, we wish to model the joint probability of all the variables $\theta, \mathbf{z}, \mathbf{w}, \phi$, that is, $p(\theta, \mathbf{z}, \mathbf{w}, \phi \mid \alpha, \beta)$. To do so, we use the chain rule of probability:

$$p(\theta, \mathbf{z}, \mathbf{w}, \phi \mid \alpha, \beta) = p(\phi \mid \beta, \alpha)p(\theta \mid \phi, \alpha, \beta)p(\mathbf{z} \mid \theta, \phi, \alpha, \beta)p(\mathbf{w} \mid \mathbf{z}, \theta, \phi, \alpha, \beta)$$

$\alpha = (\alpha_1, \ldots, \alpha_K)$ and has the following density function:

$$f(x_1, \ldots, x_K) = \frac{1}{\mathrm{B}(\boldsymbol{\alpha})} \prod_{i=1}^{K} x_i^{\alpha_i - 1}$$

where $\mathrm{B}(\boldsymbol{\alpha})$ is the multivariate beta function and each x_i satisfies $x_i \geq 0$ and $\sum x_i = 1$.

Then, taking into account that the distribution of words given a topic is determined by a *Dirichlet* distribution with parameter β, we have $p(\phi \mid \alpha, \beta) = p(\phi \mid \beta)$. Similarly, the distribution of topics within a document depends only on the hyperparameter α, and therefore $p(\theta \mid \phi, \alpha, \beta) = p(\theta \mid \alpha)$.

Moreover, given θ, the topic assignment $\mathbf{z}$ is chosen according to a *Multinomial*(θ) distribution, from which we conclude that $p(\mathbf{z} \mid \theta, \alpha, \phi, \beta) = p(\mathbf{z} \mid \theta)$, whereas the choice of a word depends only on the value of the chosen topic $\mathbf{z}$ and on the distribution given by $\phi^{(z_n)}$.

In this way, the joint distribution of the model in *LDA* is given by the following expression, which indicates how each variable depends on the preceding ones in the generative process (Carpenter 2010, p. 2):

$$p(\phi, \theta, \mathbf{z}, \mathbf{w} \mid \alpha, \beta) = p(\phi \mid \beta)p(\theta \mid \alpha)p(\mathbf{z} \mid \theta)p(\mathbf{w} \mid \mathbf{z}, \phi) \tag{2}$$

4.2 Computation of Topic Representations through Inference in LDA

As observed in the generative process of the *LDA* model, the existence of an underlying structure of latent topics organizing the content of documents is postulated. In this context, each document can be represented by a probability vector θ, which encodes the relative proportion with which each topic contributes to the generation of the observed content. In this way, it is of interest to study how the *LDA* model estimates the different parameters involved in this generation process, such as θ, the topic assignments $\mathbf{z}$, and the word distributions ϕ, from the observed text.

The inferential problem in *LDA* consists of computing the posterior distribution of the hidden variables (θ, z, ϕ) given a document $\mathbf{w}$. The objective is to determine how topics are distributed within each document and which topic generates each word. This information makes it possible to analyze new documents, measure thematic similarity, perform topic-based clustering, and label documents.

In this way, by computing the posterior distribution, one obtains (Darling 2011, p. 3):

$$p(\theta, \mathbf{z}, \phi \mid \mathbf{w}, \alpha, \beta) = \frac{p(\theta, \mathbf{z}, \mathbf{w}, \phi \mid \alpha, \beta)}{p(\mathbf{w} \mid \alpha, \beta)}$$

where

$$p(\mathbf{w} \mid \alpha, \beta) = \int \int p(\theta \mid \alpha) \prod_{i=1}^{N} \sum_{z} p(\phi \mid \beta)p(z_n \mid \theta)p(w_n \mid z_n, \phi)\, d\theta\, d\phi.$$

Expanding this expression in terms of the parameters of the generative process, we obtain:

- $p(\theta \mid \alpha)$ is a probability density function given by a *Dirichlet* distribution.
- $p(z_n = i \mid \theta) = \theta_i$, since a topic z_n is chosen for each word from a *Multinomial*(θ).

- $p(w_n \mid z_n = j, \phi) = \prod_{i=1}^{V} (\phi_i^{(j)})^{w_n^i}$, considering that each observed word $w_n = (w_n^1, w_n^2, \ldots, w_n^V)$ is represented a *one-hot encoding* [4], and a word w_n is chosen from a multinomial distribution conditioned on topic z_i.

Therefore, the equation is such that

$$p(\mathbf{w} \mid \alpha, \beta) =$$

$$\int\int \sum_{\mathbf{z}} \left[\prod_{k=1}^{K} \left(\frac{1}{B(\beta)} \prod_{w=1}^{V} \phi_{k,w}^{\beta_w - 1} \right) \prod_{d=1}^{D} \left(\frac{1}{B(\alpha)} \prod_{k=1}^{K} \theta_{d,k}^{\alpha_k - 1} \right) \prod_{d=1}^{D} \prod_{n=1}^{N_d} \theta_{d,z_{d,n}} \prod_{d=1}^{D} \prod_{n=1}^{N_d} \phi_{z_{d,n}, w_{d,n}} \right] d\theta \, d\phi$$

This expression is computationally intractable, and therefore different strategies must be found in order to carry out the computation.

4.3 Gibbs Sampling

In order to compute the posterior distribution formulated above, we shall make use of a method known as *Gibbs sampling*. The *Gibbs sampling* method belongs to the family of *Markov chain Monte Carlo* (MCMC) algorithms and provides a practical strategy for obtaining samples from a joint distribution $p(x_1, \ldots, x_m)$ when direct simulation from that distribution is difficult or intractable. As described in (Gelfand 2000, p. 1301), the central idea of the method consists in exploiting knowledge of the full conditional distributions of the components of the random vector $x = (x_1, \ldots, x_m)$ in order to construct a Markov chain whose stationary distribution is precisely the target distribution $p(x_1, \ldots, x_m)$.

In other words, Gibbs sampling makes it possible to obtain samples from the joint distribution of a random vector through an iterative process in which, at each step, the previous approximation is improved. After sufficiently many iterations, the samples obtained through this process converge to the correct joint distribution (see the stochastic foundation in [C]).

4.3.1 Algorithm

Suppose we wish to obtain samples from a random vector $x = (x_1, \ldots, x_m)$ with joint distribution $p(x_1, \ldots, x_m)$, which is difficult (or impossible) to compute directly. Nevertheless, we know the conditional distributions (Gelfand 2000, p. 1301):

- $p(x_1 \mid x_2, x_3, \ldots, x_m)$,
- $p(x_2 \mid x_1, x_3, \ldots, x_m)$,
- etc.

[4] Formally, given a category $i \in \{1, 2, \ldots, n\}$, its associated one-hot vector is:

$$\mathbf{e}_i = (0, 0, \ldots, 1, \ldots, 0) \in \mathbb{R}^n$$

where the value 1 is located in the i-th position.

Gibbs sampling is carried out according to the following steps:

1. Each x_i is initialized randomly.
2. The following procedure is iteratively executed (for $n = 1, \ldots, N$):
   - Sample $x_1^{(n+1)}$ using $p(x_1 \mid x_2^{(n)}, x_3^{(n)}, \ldots, x_m^{(n)})$.
   - Sample $x_2^{(n+1)}$ using $p(x_2 \mid x_1^{(n+1)}, x_3^{(n)}, \ldots, x_m^{(n)})$.
   - This process continues until reaching x_m.

After many iterations, the obtained samples converge to samples drawn from the random vector $(x_1, \ldots, x_m)$. Although this convergence is guaranteed in theory, there is no universal rule for determining the exact number of iterations required to achieve a sufficiently accurate approximation.

4.3.2 Inference through Gibbs Sampling

As previously stated, in *LDA* inference focuses on obtaining three important latent variables: the topic distribution for each document θ_d, the topic-word distribution $\phi^{(z)}$, and the topic assignments z_i for each word in each document.

Since, in the *Latent Dirichlet Allocation* model, the posterior distributions of the parameters θ and ϕ can be computed explicitly given the topic assignments z_i, we may integrate these latent variables out of the model. This integration leads to a simplification of the inferential process known as *collapsed Gibbs sampling*, in which the random vector sampled explicitly is the set of topic assignments $\mathbf{z} = (z_1, \ldots, z_N)$. This strategy not only reduces the dimensionality of the sampling space, but also improves the computational efficiency and convergence of the Markov chain, as discussed in (Xiao and Stibor 2010, p. 63).

More specifically, *collapsed Gibbs sampling* for *LDA* iteratively computes the conditional probability that a specific word is assigned to a given topic, assuming that the assignments of all the other topics are known, except for that of the particular word under consideration.

4.3.3 Gibbs Sampling for Topic Assignment

Given the goal of assigning topics to observed words, in *LDA* the random vector to be sampled is the assignment of latent topics to words. Following the previous notation, we wish to model the random vector

$$x = (z_1, z_2, \ldots, z_n \mid \mathbf{w}, \beta, \alpha),$$

and, therefore, the conditional distributions to be used in *Gibbs sampling* are

$$p(z_i \mid z_{-i}, \alpha, \beta, \mathbf{w}),$$

where z_{-i} denotes the assignment of all other topics except the one corresponding to word i. Following the partial calculation strategy presented in (Darling 2011, p. 6), we adapt the procedure to our framework and complete the reasoning in a fully explicit manner, considering all intermediate steps.

Computation of $p(z_i \mid z_{-i}, \alpha, \beta, \mathbf{w})$
To compute the above conditional probability, we consider the following transformations. By the rules of conditional probability, this is proportional to:

$$p(z_i \mid z_{-i}, \alpha, \beta, \mathbf{w}) = \frac{p(z_i, z_{-i}, \mathbf{w} \mid \alpha, \beta)}{p(z_{-i}, \mathbf{w} \mid \alpha, \beta)}.$$

If we marginalize with respect to the variables θ and ϕ, we obtain a description of $p(z_i, z_{-i}, \mathbf{w} \mid \alpha, \beta) = p(\mathbf{z}, \mathbf{w} \mid \alpha, \beta)$ in terms of the joint distribution of the *LDA* model described previously in (2):

$$\begin{aligned} p(\mathbf{w}, \mathbf{z} \mid \alpha, \beta) &= \int\int p(z, w, \theta, \phi \mid \alpha, \beta)\, d\theta\, d\phi \\ &= \int\int p(\phi \mid \beta)\, p(\theta \mid \alpha)\, p(z \mid \theta)\, p(w \mid \phi_z)\, d\theta\, d\phi. \end{aligned}$$

We observe that the integral may be grouped as the product of two integrals by taking into account the dependence structure between θ and ϕ (according to *Fubini's Theorem* [5]). We begin with the first integral term, $\int p(z \mid \theta)\, p(\theta \mid \alpha)\, d\theta$. Recall that, for a *Dirichlet* distribution over the topics of a document, we have:

$$p(\theta_d \mid \alpha) = \frac{\Gamma\left(\sum_{k=1}^{K} \alpha_k\right)}{\prod_{k=1}^{K} \Gamma(\alpha_k)} \prod_{k=1}^{K} \theta_{d,k}^{\alpha_k - 1}.$$

On the other hand, the multinomial distribution of the topic assignments, given a distribution θ_d, is:

$$p(\mathbf{z}_d \mid \theta_d) = \prod_{n=1}^{N_d} p(z_{d,n} \mid \theta_d) = \prod_{k=1}^{K} \theta_{d,k}^{n_{d,k}},$$

where $n_{d,k}$ is the number of words assigned to topic k in document d.

Writing this explicitly for all documents d (exploiting independence across documents) and substituting the previous equations, we obtain:

$$\begin{aligned} p(\mathbf{z} \mid \alpha) &= \prod_d \int p(\theta_d \mid \alpha)\, p(z_d \mid \theta_d)\, d\theta_d \\ &= \prod_d \int \left(\frac{\Gamma\left(\sum_{k=1}^{K} \alpha_k\right)}{\prod_{k=1}^{K} \Gamma(\alpha_k)} \prod_{k=1}^{K} \theta_{d,k}^{\alpha_k - 1} \right) \left(\prod_{k=1}^{K} \theta_{d,k}^{n_{d,k}} \right) d\theta_d. \end{aligned} \tag{3}$$

[5] *Fubini's Theorem (simplified version for integrals):* Let $a, b, c, d \in \mathbb{R}$, with $a < b$ and $c < d$, and let $f : [a, b] \times [c, d] \to \mathbb{R}$ be a Lebesgue-integrable function (that is, $f \in L^1([a, b] \times [c, d])$).
Then:

$$\iint_{[a,b]\times[c,d]} f(x, y)\, dx\, dy = \int_a^b \left(\int_c^d f(x, y)\, dy \right) dx = \int_c^d \left(\int_a^b f(x, y)\, dx \right) dy.$$

Moreover, the functions $x \mapsto \int_c^d f(x, y)\, dy$ and $y \mapsto \int_a^b f(x, y)\, dx$ are integrable.

We observe that the product contains terms with the same base $\theta_{d,k}$, and therefore the exponents may be added:

$$\theta_{d,k}^{(\alpha_k-1)} \cdot \theta_{d,k}^{n_{d,k}} = \theta_{d,k}^{n_{d,k}+\alpha_k-1},$$

so that expression (3) becomes:

$$p(\mathbf{z} \mid \alpha) = \prod_d \frac{\Gamma\left(\sum_{k=1}^K \alpha_k\right)}{\prod_{k=1}^K \Gamma(\alpha_k)} \int \prod_{k=1}^K \theta_{d,k}^{n_{d,k}+\alpha_k-1}\, d\theta_d.$$

Note that the integral involved is precisely the definition of the multinomial beta function (the multivariate generalization of the Beta function):

$$\int \prod_{k=1}^K \theta_{d,k}^{x_k-1} d\theta_d = \frac{\prod_{k=1}^K \Gamma(x_k)}{\Gamma\left(\sum_{k=1}^K x_k\right)} = B(x),$$

where the integral is taken over the entire admissible space (the simplex where $\sum_{k=1}^K \theta_{d,k} = 1$).

Using this, we clearly identify in our case that $x_k = n_{d,k} + \alpha_k$, and thus

$$p(\mathbf{z} \mid \alpha) = \prod_d \frac{\Gamma\left(\sum_k \alpha_k\right)}{\prod_k \Gamma(\alpha_k)} \frac{\prod_k \Gamma(n_{d,k}+\alpha_k)}{\Gamma\left(\sum_k (n_{d,k}+\alpha_k)\right)}.$$

Making use of the definition of the multinomial beta function, a more compact expression is obtained:

$$p(\mathbf{z} \mid \alpha) = \prod_d \frac{B(n_{d,\cdot}+\alpha)}{B(\alpha)} \tag{4}$$

with $n_{d,\cdot} + \alpha = (n_{d,1}+\alpha_1, \ldots, n_{d,K}+\alpha_K)$.

Similarly, for the second integral term:

$$\begin{aligned}\int p(\mathbf{w} \mid \phi_z) p(\phi \mid \beta) d\phi &= \int \prod_i \phi_{z_i,w_i} \frac{1}{B(\beta)} \prod_k \prod_w \phi_{k,w}^{\beta_w-1} d\phi_k \\ &= \prod_k \frac{1}{B(\beta)} \int \prod_w \phi_{k,w}^{n_{k,w}+\beta_w-1} d\phi_k = \prod_k \frac{B(n_{k,\cdot}+\beta)}{B(\beta)}. \end{aligned} \tag{5}$$

Combining the two previous equations, (4) and (5), we finally obtain:

$$p(\mathbf{w}, \mathbf{z} \mid \alpha, \beta) = \prod_d \frac{B(n_{d,\cdot}+\alpha)}{B(\alpha)} \prod_k \frac{B(n_{k,\cdot}+\beta)}{B(\beta)}.$$

Taking into account that we wish to compute

$$p(z_i = k \mid z_{-i}, \mathbf{w}, \alpha, \beta) = \frac{p(z_i = k, z_{-i}, \mathbf{w} \mid \alpha, \beta)}{p(z_{-i}, \mathbf{w} \mid \alpha, \beta)},$$

we have seen that the joint distribution with the assignment $z_i = k$ included is:

$$p(z_i = k, z_{-i}, \mathbf{w} \mid \alpha, \beta) = \prod_d \frac{B(n_{d,\cdot}+\alpha)}{B(\alpha)} \prod_k \frac{B(n_{k,\cdot}+\beta)}{B(\beta)} \tag{6}$$

whereas, on the other hand, by considering the joint distribution without the current assignment (that is, excluding z_i), the same procedure shows that:

$$p(z_{-i}, \mathbf{w} \mid \alpha, \beta) = \prod_d \frac{B(n_{d,\cdot}^{(-i)} + \alpha)}{B(\alpha)} \prod_k \frac{B(n_{k,\cdot}^{(-i)} + \beta)}{B(\beta)} \tag{7}$$

Observe that, by taking the ratio $\frac{(6)}{(7)}$, the constant terms $B(\alpha)$ and $B(\beta)$ cancel:

$$p(z_i = k \mid z_{-i}, \mathbf{w}, \alpha, \beta) = \frac{\frac{B(n_{d,\cdot}+\alpha)}{B(\alpha)} \frac{B(n_{k,\cdot}+\beta)}{B(\beta)}}{\frac{B(n_{d,\cdot}^{(-i)}+\alpha)}{B(\alpha)} \frac{B(n_{k,\cdot}^{(-i)}+\beta)}{B(\beta)}},$$

which simplifies to:

$$p(z_i = k \mid z_{-i}, \mathbf{w}, \alpha, \beta) = \frac{B(n_{d,\cdot} + \alpha)}{B(n_{d,\cdot}^{(-i)} + \alpha)} \frac{B(n_{k,\cdot} + \beta)}{B(n_{k,\cdot}^{(-i)} + \beta)}.$$

Here, $n_{d,k}$ is the number of words in document d assigned to topic k, $n_{k,w}$ is the number of times word w appears assigned to topic k, and the superscript $(-i)$ indicates that the current observation i is excluded.

Substituting the definition of the multinomial Beta function,

$$B(x) = \frac{\prod_j \Gamma(x_j)}{\Gamma\left(\sum_j x_j\right)},$$

we obtain:

$$p(z_i = k \mid z_{-i}, \mathbf{w}, \alpha, \beta) = \frac{\frac{\prod_{j=1}^K \Gamma(n_{d,j}+\alpha_j)}{\Gamma\left(\sum_{j=1}^K (n_{d,j}+\alpha_j)\right)}}{\frac{\prod_{j=1}^K \Gamma(n_{d,j}^{(-i)}+\alpha_j)}{\Gamma\left(\sum_{j=1}^K (n_{d,j}^{(-i)}+\alpha_j)\right)}} \times \frac{\frac{\prod_{w'=1}^V \Gamma(n_{k,w'}+\beta_{w'})}{\Gamma\left(\sum_{w'=1}^V (n_{k,w'}+\beta_{w'})\right)}}{\frac{\prod_{w'=1}^V \Gamma(n_{k,w'}^{(-i)}+\beta_{w'})}{\Gamma\left(\sum_{w'=1}^V (n_{k,w'}^{(-i)}+\beta_{w'})\right)}}.$$

At this point, consider the following observation. When only the current word is included:

- For the document-topic part, only the count for the specific topic k changes, that is, $n_{d,k} = n_{d,k}^{(-i)} + 1$, whereas for any other topic $j \neq k$, we have $n_{d,j} = n_{d,j}^{(-i)}$.
- For the topic-word part, only the count for the specific word w changes, that is, $n_{k,w} = n_{k,w}^{(-i)} + 1$, whereas for any other word $w' \neq w$, $n_{k,w'} = n_{k,w'}^{(-i)}$.

Analyzing the document-topic part using the previous observation:

$$\frac{\Gamma(n_{d,k}^{(-i)} + \alpha_k + 1) \prod_{j \neq k} \Gamma(n_{d,j}^{(-i)} + \alpha_j)}{\Gamma\left(\sum_j (n_{d,j}^{(-i)} + \alpha_j) + 1\right)} \times \frac{\Gamma\left(\sum_j (n_{d,j}^{(-i)} + \alpha_j)\right)}{\Gamma(n_{d,k}^{(-i)} + \alpha_k) \prod_{j \neq k} \Gamma(n_{d,j}^{(-i)} + \alpha_j)} = (*_1)$$

It is clear that the terms for $j \neq k$ cancel, so that:

$$(*_1) = \frac{\Gamma(n_{d,k}^{(-i)} + \alpha_k + 1)}{\Gamma(n_{d,k}^{(-i)} + \alpha_k)} \times \frac{\Gamma\left(\sum_j (n_{d,j}^{(-i)} + \alpha_j)\right)}{\Gamma\left(\sum_j (n_{d,j}^{(-i)} + \alpha_j) + 1\right)}.$$

Applying the Gamma function identity $\Gamma(x+1) = x\Gamma(x)$ to the numerator and denominator, we obtain:

$$(*_1) = (n_{d,k}^{(-i)} + \alpha_k)\frac{\Gamma\left(\sum_j (n_{d,j}^{(-i)} + \alpha_j)\right)}{\Gamma\left(\sum_j (n_{d,j}^{(-i)} + \alpha_j) + 1\right)} = \frac{(n_{d,k}^{(-i)} + \alpha_k)}{\sum_j (n_{d,j}^{(-i)} + \alpha_j)}. \tag{8}$$

Similarly, analyzing the topic-word part and simplifying by cancellation, one obtains:

$$\begin{aligned}
&\frac{\Gamma(n_{k,w}^{(-i)} + \beta_w + 1)\prod_{w' \neq w}\Gamma(n_{k,w'}^{(-i)} + \beta_{w'})}{\Gamma\left(\sum_{w'}(n_{k,w'}^{(-i)} + \beta_{w'}) + 1\right)} \times \frac{\Gamma\left(\sum_{w'}(n_{k,w'}^{(-i)} + \beta_{w'})\right)}{\Gamma(n_{k,w}^{(-i)} + \beta_w)\prod_{w' \neq w}\Gamma(n_{k,w'}^{(-i)} + \beta_{w'})} \\
&= \frac{\Gamma(n_{k,w}^{(-i)} + \beta_w + 1)}{\Gamma(n_{k,w}^{(-i)} + \beta_w)} \times \frac{\Gamma\left(\sum_{w'}(n_{k,w'}^{(-i)} + \beta_{w'})\right)}{\Gamma\left(\sum_{w'}(n_{k,w'}^{(-i)} + \beta_{w'}) + 1\right)} = (*_2).
\end{aligned}$$

Applying once again the Gamma function identity, one finally obtains:

$$(*_2) = \frac{(n_{k,w}^{(-i)} + \beta_w)}{\sum_{w'}(n_{k,w'}^{(-i)} + \beta_{w'})}. \tag{9}$$

Combining both results explicitly, (8) and (9), we obtain the well-known equation of *collapsed Gibbs sampling* in *LDA* (Darling 2011, p. 6):

$$p(z_i = k \mid z_{-i}, \mathbf{w}, \alpha, \beta) \propto \frac{(n_{d,k}^{(-i)} + \alpha_k)}{\sum_j (n_{d,j}^{(-i)} + \alpha_j)} \times \frac{(n_{k,w}^{(-i)} + \beta_w)}{\sum_{w'}(n_{k,w'}^{(-i)} + \beta_{w'})}.$$

4.4 Point Estimation of the Parameters θ_d and ϕ_k

Once the latent topic assignments to words have been obtained through *collapsed Gibbs sampling*, it becomes possible to derive point estimates of the model parameters that describe the vector representation of documents and the distribution of topics. In this section, the explicit expressions for estimating the distributions θ_d and ϕ_k are presented on the basis of the topic assignments to words, together with the reasoning that leads to these well-known expressions (Heinrich 2005, p. 23).

4.4.1 Point Estimation of the Parameter θ_d

In *LDA* we assume that, for each document d, the topic distribution θ_d follows a *Dirichlet* distribution with parameter $\alpha = (\alpha_1, \ldots, \alpha_K)$:

$$\theta_d \sim Dirichlet(\alpha).$$

This means that the density of $\theta_d = (\theta_{d,1}, \ldots, \theta_{d,K})$ is:

$$p(\theta_d \mid \alpha) = \frac{\Gamma\left(\sum_{k=1}^K \alpha_k\right)}{\prod_{k=1}^K \Gamma(\alpha_k)} \prod_{k=1}^K \theta_{d,k}^{\alpha_k - 1} \tag{10}$$

subject to the condition that $\sum_{k=1}^{K} \theta_{d,k} = 1$.

Each word $w_{d,i}$ in the document is generated from a multinomial distribution conditioned on θ_d, and for each assignment of topics to words $z_{d,i}$ we know that $p(z_{d,i} = k \mid \theta_d) = \theta_{d,k}$.

Recalling that we have defined $n_{d,k}$ as the number of times topic k has been assigned in document d,

$$n_{d,k} = \sum_{i=1}^{N_d} \mathbf{1}_{\{z_{d,i}=k\}},$$

the likelihood for the set of assignments z_d, previously obtained by Gibbs sampling, is

$$p(z_d \mid \theta_d) = \prod_{i=1}^{N_d} \theta_{d,z_{d,i}} = \prod_{k=1}^{K} \theta_{d,k}^{n_{d,k}}. \tag{11}$$

To obtain the posterior distribution of θ_d given the assignments z_d, we apply *Bayes' rule*:

$$p(\theta_d \mid z_d, \alpha) \propto p(z_d \mid \theta_d)\, p(\theta_d \mid \alpha).$$

Substituting expressions (10) and (11), we obtain:

$$p(\theta_d \mid z_d, \alpha) \propto \left[\prod_{k=1}^{K} \theta_{d,k}^{n_{d,k}}\right] \left[\frac{\Gamma\left(\sum_{k=1}^{K} \alpha_k\right)}{\prod_{k=1}^{K} \Gamma(\alpha_k)} \prod_{k=1}^{K} \theta_{d,k}^{\alpha_k - 1}\right].$$

The constant term $\frac{\Gamma\left(\sum_{k=1}^{K} \alpha_k\right)}{\prod_{k=1}^{K} \Gamma(\alpha_k)}$ does not depend on θ_d, and can therefore be absorbed into the normalization constant. Thus we obtain:

$$p(\theta_d \mid z_d, \alpha) \propto \prod_{k=1}^{K} \theta_{d,k}^{n_{d,k} + \alpha_k - 1}.$$

The resulting form is precisely the density function of a *Dirichlet* distribution with updated parameters:

$$\theta_d \mid z_d, \alpha \sim Dirichlet(n_{d,1} + \alpha_1,\, n_{d,2} + \alpha_2,\, \ldots,\, n_{d,K} + \alpha_K).$$

To obtain a point estimate of θ_d, we may compute the expectation of the posterior *Dirichlet* distribution. A basic property of a *Dirichlet* distribution is that, if

$$\theta \sim Dirichlet(\gamma_1, \ldots, \gamma_K),$$

then for each component:

$$E[\theta_k] = \frac{\gamma_k}{\sum_{j=1}^{K} \gamma_j}. \tag{12}$$

Applied to our case, the parameters are $\gamma_k = n_{d,k} + \alpha_k$. Therefore, the point estimate of the probability that document d addresses topic k is (Heinrich 2005, p. 23):

$$\hat{\theta}_{d,k} = E[\theta_{d,k} \mid z_d, \alpha] = \frac{n_{d,k} + \alpha_k}{\sum_{j=1}^{K} (n_{d,j} + \alpha_j)}.$$

4.4.2 Point Estimation of the Parameter ϕ_k

In *LDA* we assume that, for each topic k, the word distribution is modeled by a *Dirichlet* distribution with parameters $\beta = (\beta_1, \beta_2, \ldots, \beta_{\mathcal{V}})$, that is:

$$\phi_k \sim Dirichlet(\beta).$$

The density for $\phi_k = (\phi_{k,1}, \phi_{k,2}, \ldots, \phi_{k,\mathcal{V}})$ is:

$$p(\phi_k \mid \beta) = \frac{1}{B(\beta)} \prod_{w=1}^{|\mathcal{V}|} \phi_{k,w}^{\beta_w - 1},$$

where $|\mathcal{V}|$ is the size of the vocabulary, and the multinomial Beta function is defined as:

$$B(\beta) = \frac{\prod_{w=1}^{V} \Gamma(\beta_w)}{\Gamma\left(\sum_{w=1}^{V} \beta_w\right)}.$$

If we consider that, for each word assigned to topic k, a word w is observed, then the likelihood of a set of words assigned to topic k is a multinomial distribution. Let $n_{k,w}$ denote the number of times word w is assigned to topic k. The likelihood for topic k is:

$$p(\mathbf{w}_k \mid \phi_k) = \prod_{w=1}^{V} \phi_{k,w}^{n_{k,w}},$$

where $\mathbf{w}_k$ represents all words assigned to topic k.

To obtain the posterior distribution of ϕ_k given the observations $\mathbf{w}_k$ and the counts $n_{k,w}$, we apply *Bayes' rule*:

$$p(\phi_k \mid \mathbf{w}_k, \beta) \propto p(\mathbf{w}_k \mid \phi_k)\, p(\phi_k \mid \beta).$$

Substituting the previous expressions, we obtain:

$$p(\phi_k \mid \mathbf{w}_k, \beta) \propto \left[\prod_{w=1}^{V} \phi_{k,w}^{n_{k,w}}\right] \left[\frac{1}{B(\beta)} \prod_{w=1}^{V} \phi_{k,w}^{\beta_w - 1}\right].$$

Since the term $\frac{1}{B(\beta)}$ does not depend on ϕ_k, we may write:

$$p(\phi_k \mid \mathbf{w}_k, \beta) \propto \prod_{w=1}^{V} \phi_{k,w}^{n_{k,w} + \beta_w - 1}.$$

We observe that this is the functional form of a *Dirichlet* distribution with updated parameters:

$$\phi_k \mid \mathbf{w}_k, \beta \sim Dirichlet(n_{k,1} + \beta_1,\ n_{k,2} + \beta_2,\ \ldots,\ n_{k,V} + \beta_V).$$

To obtain a point estimate of ϕ_k, we use the expected value of a *Dirichlet* distribution given in (12). In our case, $\gamma_w = n_{k,w} + \beta_w$. Therefore, the point estimate for the probability that, given topic k, word w is generated is (Heinrich 2005, p. 23):

$$\hat{\phi}_{k,w} = \frac{n_{k,w} + \beta_w}{\sum_{w'=1}^{V} (n_{k,w'} + \beta_{w'})}.$$

This expression provides the word distribution for each topic k on the basis of the observed topic assignments and the prior β.

4.5 Training an LDA Model

Once an iteration of Gibbs sampling has been performed, the parameters α, which control the topic distribution per document, and β, which control the word distribution per topic, are usually updated (Minka 2000, p. 1).

These parameters may be adjusted by maximizing the marginal distribution of the topic assignments previously obtained in (4), defined by

$$p(\mathbf{z} \mid \boldsymbol{\alpha}) = \prod_{d=1}^{D} \frac{B(n_{d,\cdot} + \boldsymbol{\alpha})}{B(\boldsymbol{\alpha})}.$$

Considering the log-likelihood of the previous expression, one obtains

$$\log p(\mathbf{z} \mid \boldsymbol{\alpha}) = \sum_{d=1}^{D} \log \frac{B(n_{d,\cdot} + \boldsymbol{\alpha})}{B(\boldsymbol{\alpha})}.$$

Taking into account that

$$\log B(\boldsymbol{\alpha}) = \sum_{k=1}^{K} \log \Gamma(\alpha_k) - \log \Gamma\left(\sum_{k=1}^{K} \alpha_k\right),$$

it follows that

$$\begin{aligned}\log p(\mathbf{z} \mid \boldsymbol{\alpha}) = &\sum_{d=1}^{D} \left[\sum_k \log \Gamma(n_{d,k} + \alpha_k) - \log \Gamma\left(\sum_k (n_{d,k} + \alpha_k)\right)\right] \\ &- D\left[\sum_k \log \Gamma(\alpha_k) - \log \Gamma\left(\sum_k \alpha_k\right)\right].\end{aligned}$$

Differentiating the previous equation with respect to α_k and setting the derivative equal to zero yields the following result:

$$\frac{\partial}{\partial \alpha_k} \log p(\mathbf{z} \mid \boldsymbol{\alpha}) = \sum_d \left[\psi(n_{d,k} + \alpha_k) - \psi\left(\sum_j (n_{d,j} + \alpha_j)\right)\right] - D\left[\psi(\alpha_k) - \psi\left(\sum_j \alpha_j\right)\right] = 0 \tag{13}$$

where $\psi(x)$ is the well-known digamma function defined by $\psi(x) = \frac{\partial}{\partial x} \log(\Gamma(x))$.

The previous expression does not admit a closed-form solution, which makes it necessary to consider alternative methods for solving it. Nevertheless, the solution may be obtained through numerical procedures, such as the Newton–Raphson method. In this regard, (Minka 2000, p. 2) proposes a reformulation of the equation derived above as a fixed-point equation and shows that, for expression (13), the Newton–Raphson method exhibits local convergence under certain conditions. Analogously, the same reasoning may be applied to the parameter β.

5 Methodology

The present study focuses on evaluating, from a qualitative perspective, how vector representations of text affect binary classification tasks through the three vector representations discussed above: *TF-IDF*, *LSA*, and *LDA*.

In order to comparatively evaluate the performance of different text vector representation techniques, three document corpora have been selected to which these representations will be applied. Subsequently, a classifier based on *Gradient Boosting* [A.3] will be trained, using the area under the ROC curve (*AUC-ROC*) [A.1.6] as the main evaluation metric on a test set.

All experiments will be carried out on the same computer, equipped with a 64-bit processor, 1.20 GHz, 8 GB of RAM, and 4 cores.

5.1 Spams dataset

First, we consider a dataset of *spam* emails (Hopkins et al. 1999), consisting of a collection of short documents with relatively low lexical richness. The content of the messages is characterized by the presence of recurring phrases and specific keywords such as *free*, *win*, or *call*, which are systematically repeated across *spam* messages. This textual structure, which is more centered on specific lexical units than on semantically differentiated discursive styles, makes representations such as *TF-IDF*, which preserve the original granularity of the vocabulary, particularly suitable in this context. The documents are labeled with a 1 if they correspond to a *spam* message, and with a 0 otherwise.

Let us first consider a *Gradient Boosting* model with depth fixed at 6 and number of trees equal to 500. Using the *AUC-ROC* metric under 5-fold cross-validation [A.2.1], we obtain the following results [B]:

Representation	**Mean AUC-ROC**	**Execution time (approx.)**	***k***
TF-IDF	0.9826	53 s	10000
LSA	0.9818	10 s	100
LDA	0.9657	20 s	10

Table 1: Comparison of the performance of the TF-IDF, LSA, and LDA vector representation models on the *Spams Dataset*

The results show that the *TF-IDF*-based representation achieves the best performance (mean *AUC-ROC* of 0.9826), followed by *LSA* (0.9818) and, finally, *LDA* (0.9657). These differences can be justified intuitively in light of what is known about these techniques. Through singular value decomposition (*SVD*), *LSA* is able to capture latent semantic structures in the document space, identifying correlations between terms that may be useful for classification even when they do not explicitly co-occur. This dimensionality reduction (from 10000 to only 100 dimensions) not only preserves the relevant information, but may also reduce statistical noise. By contrast, *TF-IDF* preserves the original

granularity of the vocabulary, which can be beneficial for models such as *Gradient Boosting*, which perform well in high-dimensional spaces and can exploit nonlinear relationships between specific terms associated with *spam*. Nevertheless, this high dimensionality entails a significantly higher computational cost. Finally, *LDA*, which models documents as distributions over topics, provides a compact thematic representation (only 10 dimensions) and achieves good performance with an intermediate execution time. This suggests that *LDA* may be an efficient option in scenarios where speed and interpretability of the representation are priorities, even though its classification performance is slightly lower [6].

5.2 Twitter dataset

Let us consider the *Sentiment140* document corpus Go, Bhayani, and Huang 2009. This corpus of 1.5 million records offers remarkable semantic richness, capturing the linguistic and expressive diversity characteristic of communication on Twitter. For reasons of computational efficiency, we will work with a subset of documents from this corpus. The tweets reflect a wide range of topics and styles, from opinions about products to comments on current events, thereby providing a varied and realistic context for training and evaluating text vector representation models. Each tweet is labeled with a binary value: 1 if it expresses negative sentiment, and 0 otherwise.

Let us now consider a slightly different situation from the previous result. In order to evaluate the degree of fit of the model under these conditions, a logistic regression has been used as a simple linear approximation, in contrast to the *Gradient Boosting* model used previously. The results obtained under this experimental framework are detailed below [B]:

Representation	**Mean AUC-ROC**	**Execution time (approx.)**	$\boldsymbol{k}$
TF-IDF	0.8239	3 s	10000
LSA	0.7631	12 s	100
LDA	0.6040	100 s	10

Table 2: Comparison of the performance of the TF-IDF, LSA, and LDA vector representation models on the *Twitter Dataset*

In this case, representation through *TF-IDF* again maintains slightly superior performance (mean *AUC-ROC* of 0.8239), followed by *LSA* (0.7631), whereas *LDA* shows substantially lower performance (0.6040). *TF-IDF* considers a representation in a 10000-dimensional space, capturing all those patterns that may be relevant or make a difference for classification. *LSA*, by contrast, projects the documents into a lower-dimensional latent space ($k = 100$), capturing semantic correlations between terms while eliminating components that may still be relevant for classification. Nevertheless, in this setting, *LSA* is capable of capturing the semantics present in the corpus and delivering performance relatively close to that of *TF-IDF*. Finally, it is observed that in cases where the *LDA*

[6] Although the *AUC-ROC* values obtained with *TF-IDF* are slightly higher than those obtained with *LSA*, statistical analysis using a paired-samples t-test (assuming normality verified by a QQ-plot) does not allow one to conclude that this difference is statistically significant at the 95% confidence level.

model does not detect a clear thematic structure among the documents, its performance drops notably. Regarding execution time, the *TF-IDF* representation proves to be the most efficient (3 s), followed by *LSA* (12 s). By contrast, *LDA* requires a longer execution time, attributable to its greater computational complexity. This time difference was offset in the previous example, where a *Gradient Boosting* model was trained on the low-dimensional vector representation provided by *LDA*, thereby favoring rapid training of this more complex model.

5.3 20 Newsgroups Dataset

The *20 Newsgroups* dataset (Lang 1995) is a corpus composed of approximately 18,000 documents classified into 20 distinct thematic categories. This corpus is characterized by a high degree of internal semantic cohesion among the documents within the same category, with specific vocabulary and similar discursive patterns, as well as a clear thematic separation between groups, which makes it a particularly suitable resource for vector representation by means of *LDA*. For the experiment carried out, each document has been labeled with a 1 if it belongs to a category related to sports, and with a 0 otherwise.

Returning to the *Gradient Boosting* model with 500 trees and depth 6, under the same control conditions, the following results are obtained [B]:

Representation	**Mean AUC-ROC**	**Execution time (approx.)**	***k***
TF-IDF	0.9915	650 s	10000
LSA	0.9902	120 s	100
LDA	0.9743	520 s	20

Table 3: Comparison of the performance of the TF-IDF, LSA, and LDA vector representation models on the *20 Newsgroups Dataset*

The results obtained show that, in this particular case, the *TF-IDF*-based vector representation continues to provide the best predictive performance. Nevertheless, semantic reduction techniques such as *LDA* and *LSA* exhibit very competitive behavior in less time. This behavior can be explained by two structural properties of the dataset used. First, the clear thematic separation among the documents favors methods such as *LDA*, which explicitly model topic distributions, allowing them to capture the most representative latent dimensions even while working with a very small number of dimensions ($k = 20$). Second, the strong semantic coherence within each thematic group facilitates the identification by *LSA* of recurrent lexical patterns through linear combinations of latent concepts. In this way, *LSA* emerges as a compact and computationally efficient alternative (120 s) in environments where the semantic relationship between words and documents is clear, whereas *LDA* stands out for its ability to represent the underlying thematic structure with few parameters, at the cost of a higher execution time (520 s) [7].

[7] Although the *AUC-ROC* values obtained with *LDA* are slightly lower than those obtained with *TF-IDF* or *LSA*, statistical analysis using a paired-samples t-test (assuming normality verified by a *QQ*-plot) allows one to conclude that this difference is statistically significant at the 95% confidence level.

5.4 Summary of Results

Although the *TF-IDF*-based vector representation has generally offered the best performance in terms of binary classification according to *AUC-ROC*, the results obtained show that both *LSA* and *LDA* may constitute competitive alternatives, especially in scenarios where computational efficiency and compactness of the representation are relevant considerations.

In particular, it has been observed that, in contexts involving documents with strong internal semantic coherence and explicit thematic separation, as in the case of the *20 Newsgroups* corpus, the latent representations provided by *LSA* and *LDA* deliver highly competitive performance comparable to the representation given by *TF-IDF*, benefiting from their ability to capture global semantic structures.

On the other hand, when working with large corpora and extensive vocabularies, as in the case of the Twitter dataset, although *LDA* and *LSA* are computationally more expensive to train than the construction of a *TF-IDF* matrix, this initial cost is compensated when a complex classifier such as *Gradient Boosting* is subsequently applied, thanks to the significant reduction in dimensionality. This can be illustrated in the following comparison of training times among *LDA*+*Gradient Boosting*, *TF-IDF*+*Gradient Boosting*, and *LSA*+*Gradient Boosting*:

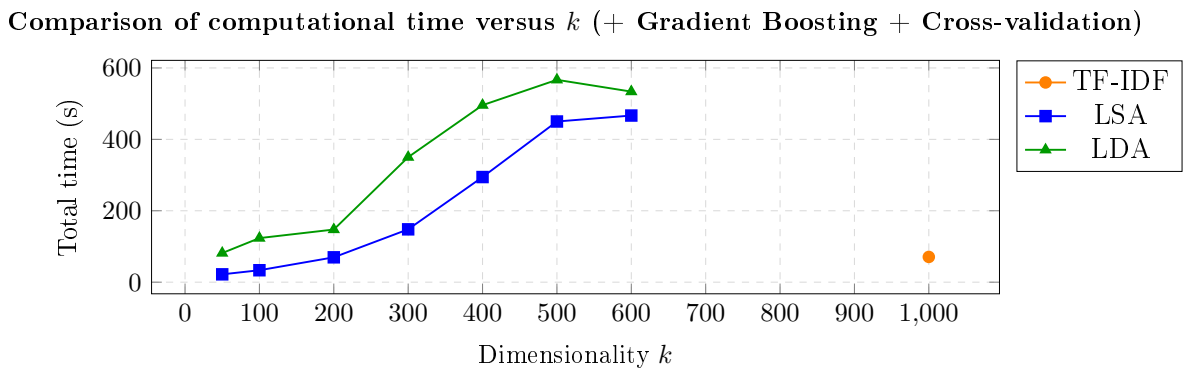

Figure 4: Comparison of computational time as a function of dimensionality k for the *TF-IDF*, *LSA*, and *LDA* methods trained on the *Spams* corpus.

The previous graph shows that the computational complexity of *LDA* is higher than that of *LSA*, which is consistent with the findings reported by various authors. Whereas *LSA*, through truncated *SVD*, has complexity $\mathcal{O}(mnk)$ (Feng, Yu, and Li 2018, p. 2), the complexity of *LDA* is given by $\mathcal{O}(kND)$, where N is the total number of words appearing in the corpus (He et al. 2017, p. 5). This difference is also reflected empirically in the execution times observed in this work.

In parallel, *LSA* has shown very satisfactory behavior in contexts where the texts exhibit notable semantic richness, as in the case of the tweet corpus with high lexical diversity or the thematically structured and semantically related *20 Newsgroups* corpus. In this same context, *LDA* has shown performance comparable to that of the other two vector representations studied, although always maintaining slightly lower performance,

sacrificing predictive accuracy in exchange for much greater interpretability in the representation. It has also been observed that the performance of *LDA* decreases notably when the document collection does not exhibit a clear underlying thematic structure, since *Gibbs sampling* fails to generate a consistent vector representation, thereby revealing its sensitivity to noise.

Finally, it is important to emphasize that both *LSA* and *LDA* drastically reduce memory cost, since their vector representations operate in latent spaces of much lower dimensionality than the original space, which may be decisive in environments with computational constraints.

Let us consider the following summary table:

Table 4: Practical comparison of text vector representations

Criterion	**TF-IDF**	**LSA**	**LDA**
Interpretability	Medium	Medium	High
Computational efficiency (+ Gradient Boosting)	Low	High	Medium
Robustness to noise	High	High	Low
Semantic capacity	Low	High	Medium

6 Conclusions

This work has provided an analysis of the main techniques for vector representation of text, placing particular emphasis on their mathematical foundations and their practical application to classification problems. The initial motivation, centered on the need to transform textual documents into numerical representations suitable for machine learning models, has guided the development of a dual approach: theoretical and empirical.

From a theoretical point of view, the structure of the *TF-IDF*, *LSA*, and *LDA* techniques has been formalized, showing how both documents and terms are represented in a meaningful way. Concepts such as the latent semantic space, singular value decomposition, and inference through Gibbs sampling have been examined in detail, highlighting their implications for the understanding and manipulation of textual content.

At a practical level, it has been shown that these representations are not only competitive alternatives in terms of classifier performance on prediction tasks, but also offer advantages in terms of dimensionality reduction, interpretability, and computational efficiency. The empirical experiments carried out on several datasets have made it possible to compare the behavior of the different techniques, showing that representations based on *LSA* and *LDA* can constitute real and viable alternatives for document representation in practical contexts, although it is important to emphasize that they depend strongly on the semantic structure exhibited by the corpus.

Overall, this study has highlighted the relevance of vector representations as a bridge between the information contained in text and the algorithms used in predictive models, reinforcing the idea that a careful mathematical formulation can contribute decisively to improvements in text processing.

References

Berry, Michael W and Murray Browne (2005). *Understanding search engines: mathematical modeling and text retrieval*. SIAM.

Blei, DM, AY Ng, and MI Jordan (2003). "Latent Dirichlet Allocation". In: *Journal of Machine Learning Research* 3.

Carpenter, Bob (2010). "Integrating out multinomial parameters in latent dirichlet allocation and naive bayes for collapsed gibbs sampling". In: *Rapport Technique* 4, p. 464.

Darling, William M (2011). "A theoretical and practical implementation tutorial on topic modeling and gibbs sampling". In: *Proceedings of the 49th annual meeting of the association for computational linguistics: Human language technologies*, pp. 642–647.

Deerwester, Scott et al. (1990). "Indexing by latent semantic analysis". In: *Journal of the American society for information science* 41.6, pp. 391–407.

Dongarra, Jack et al. (2018). "The singular value decomposition: Anatomy of optimizing an algorithm for extreme scale". In: *SIAM review* 60.4, pp. 808–865.

Eckart, Carl and Gale Young (1936). "The approximation of one matrix by another of lower rank". In: *Psychometrika* 1.3, pp. 211–218.

Feng, Xu, Wenjian Yu, and Yaohang Li (2018). "Faster matrix completion using randomized SVD". In: *2018 IEEE 30th International conference on tools with artificial intelligence (ICTAI)*. IEEE, pp. 608–615.

Ferrante, Marco and Xavier Bardina (2023). *An excursion into Markov chains*. Springer.

Gelfand, Alan E (2000). "Gibbs sampling". In: *Journal of the American statistical Association* 95.452, pp. 1300–1304.

Go, Alec, Richa Bhayani, and Lei Huang (2009). *Twitter Sentiment Classification using Distant Supervision*. `http://help.sentiment140.com/for-students`. Accessed: 2024-05-15.

He, Junxian et al. (2017). "Efficient correlated topic modeling with topic embedding". In: *Proceedings of the 23rd ACM SIGKDD international conference on knowledge discovery and data mining*, pp. 225–233.

Heinrich, Gregor (2005). *Parameter estimation for text analysis*. Tech. rep. Citeseer.

Hopkins, Mark et al. (1999). *Spambase Data Set*. `https://archive.ics.uci.edu/ml/datasets/spambase`. Accessed: 2024-05-15.

Horasan, Fahrettin et al. (2019). "Alternate Low-Rank Matrix Approximation in Latent Semantic Analysis". In: *Scientific Programming* 2019.1, p. 1095643.

James, Gareth et al. (2023). *An Introduction to Statistical Learning with Applications in Python*.

Jelodar, Hamed et al. (2019). "Latent Dirichlet allocation (LDA) and topic modeling: models, applications, a survey". In: *Multimedia tools and applications* 78, pp. 15169–15211.

Khim, Suyeon (2007). "The Frobenius–Perron theorem". PhD thesis. Doctoral Dissertation, PhD thesis, The University of Chicago.

Landauer, Thomas K and Susan T Dumais (1997). "A solution to Plato's problem: The latent semantic analysis theory of acquisition, induction, and representation of knowledge." In: *Psychological review* 104.2, p. 211.

Lang, Ken (1995). *20 Newsgroups Dataset*. `http://qwone.com/~jason/20Newsgroups/`. Accessed: 2024-05-15.

Leydesdorff, Loet and Kasper Welbers (2011). "The semantic mapping of words and co-words in contexts". In: *Journal of informetrics* 5.3, pp. 469–475.

Martin, Dian I and Michael W Berry (2007). "Mathematical foundations behind latent semantic analysis". In: *Handbook of latent semantic analysis*, pp. 35–56.

Meyer, Carl D. (2000). *Matrix Analysis and Applied Linear Algebra.* Society for Industrial and Applied Mathematics (SIAM).

Minka, Thomas (2000). *Estimating a Dirichlet distribution.*

Rebeschini, Patrick (2018). "Chapter 5: Gibbs Sampling". In: *Advanced Simulation Methods.* SC5 Lecture Notes. University of Oxford.

Schütze, Hinrich, Christopher D Manning, and Prabhakar Raghavan (2008). *Introduction to information retrieval.* Vol. 39. Cambridge University Press Cambridge.

Valle-Lisboa, Juan C and Eduardo Mizraji (2007). "The uncovering of hidden structures by latent semantic analysis". In: *Information sciences* 177.19, pp. 4122–4147.

Xiao, Han and Thomas Stibor (2010). "Efficient collapsed gibbs sampling for latent dirichlet allocation". In: *Proceedings of 2nd asian conference on machine learning.* JMLR Workshop and Conference Proceedings, pp. 63–78.

A Appendix

This appendix presents a rigorous and formal exposition, from a mathematical point of view, of the foundations of machine learning, with particular attention to supervised learning. It also includes a detailed theoretical description of the functioning and underlying principles of the *Gradient Boosting*-based classifier, highlighting its algorithmic aspects and its analytical justification (based on (James et al. 2023)).

A.1 Statistical Learning

A.1.1 Introduction

Artificial intelligence (AI) is a field of computer science devoted to developing systems and computer programs capable of performing tasks that normally require human intelligence. These tasks include a wide range of activities, such as recognizing patterns in data, making logical decisions, solving complex problems, understanding and using natural language, and learning from experience through data.

Machine learning (ML) is a subfield of AI that focuses on the development of algorithms that allow machines to learn from data without being explicitly programmed. Instead of following specific instructions, ML algorithms identify patterns and relationships in the data in order to improve their performance on a specific task.

These techniques can be classified into supervised learning and unsupervised learning. In general terms, supervised learning consists of training a model with labeled data, where the algorithm learns to predict an output from one or more inputs.

On the other hand, unsupervised learning works with unlabeled data, in which there is no predefined output. Instead, the algorithm identifies hidden patterns and structures in the data.

A.1.2 Supervised Learning

Let $X \subseteq \mathbb{R}^d$ be the space of explanatory variables (or independent variables), and let $Y \subseteq \mathbb{R}$ be the space of target variables (or dependent variables).

Definició 11. *A supervised learning problem is defined as the task of approximating an unknown function $f : X \to Y$ from a dataset $\{(x_i, y_i)\}_{i=1}^N$, with $x_i \in X$ and $y_i \in Y$.*

The existence of a map $f : X \to Y$ describing the relationship between the independent variables and the dependent variable is assumed. The objective of supervised learning is to find an estimated function $\hat{f}$ that approximates f as well as possible according to a given error criterion.

$$\hat{f} : X \longrightarrow Y$$

such that:

$$\hat{f}(x_i) \approx y_i, \quad \forall (x_i, y_i).$$

In general, one does not seek to determine the best function in absolute terms, since this would constitute an irresolvable problem. Instead, a hypothesis space H is defined, representing a family of functions with a common structure. This family of functions is parameterized by a set of parameters ϕ, so that each function in H can be expressed as:

$$H = \{f_\phi : X \to Y \mid \phi \in \Phi\}.$$

The specific values of the parameters ϕ determine a particular function f_ϕ within the family H. Thus, the objective is to find the best configuration of parameters ϕ^* such that the resulting function $f_{\phi^*} \in H$ satisfies

$$f_{\phi^*}(x_i) \approx y_i, \quad \forall (x_i, y_i).$$

A.1.3 Classification and Regression

Definició 12. *Let $X \subseteq \mathbb{R}^d$ be the feature space, and let Y be a finite and predefined set of classes:*

$$Y = \{c_1, c_2, \ldots, c_K\}, \quad K \in \mathbb{N}.$$

A classification problem consists of finding a decision function:

$$f : X \to Y,$$

such that, for each observation $x_i \in X$, it assigns it to a class $y_i \in Y$.

Definició 13. *Let $X \subseteq \mathbb{R}^d$ be the space of explanatory variables, and let $Y \subseteq \mathbb{R}$ be the space of the target variable (or dependent variable).*

A regression problem consists of finding a function $f : X \to Y$ that makes it possible to predict a continuous numerical value y_i for each observation $x_i \in X$.

A.1.4 Loss Function

To evaluate the degree of fit of a model to the data $\{(x_i, y_i)\}_{i=1}^{N}$, a cost function or loss function $L : Y \times Y \longrightarrow \mathbb{R}^+$ is defined, which quantifies the discrepancy between the model's predictions and the expected values.

Supervised learning is deeply related to conditional probability, since many supervised models and algorithms are based on probabilistic principles, where $P(Y \mid X)$ plays a fundamental role in the construction of the model.

For example, given a dataset with features X and labels Y, the objective is to predict Y from X, that is, to estimate the conditional probability of the output variable given a particular input X.

In many supervised models, one seeks to model $P(Y \mid X)$, which represents the probability that the variable Y takes a certain value given the feature vector X.

A common method for estimating the model parameters θ is maximum likelihood estimation. This consists of finding the values of θ that maximize the conditional probability of observing the labels Y given the predictions of the model.

We are interested in this probability being high. Thus, we maximize the joint likelihood over all observations:

$$\hat{\phi} = \arg\max_{\phi} \left(\prod_{i=1}^{n} P(y_i \mid f_\phi(x_i)) \right)$$

This constitutes the method of maximum likelihood.

In a probabilistic context, likelihood is a function that measures how probable it is to observe a dataset given a set of model parameters θ. If we have a training set $\{(X_i, Y_i)\}$, where X_i are the features and Y_i are the labels, the likelihood for a model $P(Y \mid X; \theta)$ is defined as:

$$L(\theta) = P(Y_1, \ldots, Y_n \mid X_1, \ldots, X_n; \theta)$$

Assuming conditional independence among the observations:

$$L(\theta) = \prod_{i=1}^{n} P(Y_i \mid X_i; \theta)$$

To optimize more easily, one often uses the log-likelihood, that is, the natural logarithm of the likelihood function:

$$\log L(\theta) = \sum_{i=1}^{n} \log P(Y_i \mid X_i; \theta)$$

This transformation simplifies the calculations, since it converts products into sums, which is useful in optimization and avoids numerical problems arising from multiplying many small probabilities. Thus, maximizing the log-likelihood is equivalent to maximizing the likelihood.

In optimization problems, it is common to reformulate the task as a minimization rather than a maximization. To do so, the negative of the log-likelihood function is considered, thus obtaining the *negative log-likelihood*:

$$\hat{\phi} = -\arg\min_{\phi} \left(P(Y_i \mid X_i; \theta)\right)$$

This reformulation makes it possible to interpret model optimization as a minimization problem, thereby facilitating its solution through standard optimization techniques.

The cost or loss function is used during the model training process. This function quantifies the error between the model's predictions and the true values, and constitutes the criterion that the model attempts to minimize in order to adjust its parameters and thereby optimize the accuracy of its predictions.

A.1.5 Metrics

Definició 14. *Let $f_\theta : \mathcal{X}' \to \mathcal{Y}'$ be a machine learning model parameterized by θ, trained to solve a regression or classification problem.*

An evaluation metric is a function $M : \mathcal{Y}' \times \mathcal{Y}' \to \mathbb{R}$ that quantifies the performance of the model on a test dataset $\{(x_i, y_i)\}_{i=1}^{N}$, where $x_i \in \mathcal{X}'$ are the inputs and $y_i \in \mathcal{Y}'$ are the true outputs.

In binary classification problems, where the objective is to predict one of two possible classes (for example, "positive" or "negative"), there exist several fundamental metrics for evaluating model performance. These metrics include accuracy (*Accuracy*), precision (*Precision*), the true positive rate (*True Positive Rate*, also known as *Recall* or *TPR*), the false positive rate (*False Positive Rate*, *FPR*), and the confusion matrix (*Confusion Matrix*).

These metrics provide a more detailed evaluation of the model's behavior, allowing for a deeper analysis than simply considering the percentage of correct or incorrect predictions.

Definició 15. *Let there be a binary classification problem with an output variable Y taking values in the set $\{0, 1\}$, where 1 represents the positive class and 0 the negative class. Given a classification model that assigns a prediction $\hat{y}$ to each observation, the confusion matrix is defined as a table summarizing the correspondence between the model's predictions and the true classes.*

Formally, the confusion matrix is composed of the following elements:

- *True Positives (TP):* Number of instances for which $\hat{Y} = 1$ and $Y = 1$.
- *False Positives (FP):* Number of instances for which $\hat{Y} = 1$ and $Y = 0$.

- *True Negatives (TN):* Number of instances for which $\hat{Y} = 0$ and $Y = 0$.
- *False Negatives (FN):* Number of instances for which $\hat{Y} = 0$ and $Y = 1$.

The confusion matrix is frequently represented in the following tabular form:

	True Class $Y = 1$	True Class $Y = 0$
Prediction $\hat{Y} = 1$	TP	FP
Prediction $\hat{Y} = 0$	FN	TN

This matrix makes it possible to evaluate model performance in a detailed way, providing key information about the distribution of correct and incorrect predictions.

Definició 16. *Accuracy measures the proportion of correct predictions, considering both true positives (TP) and true negatives (TN), relative to the total number of predictions made. It is defined mathematically as:*

$$Acc = \frac{TP + TN}{TP + FP + TN + FN}$$

It is important to note that this metric may present limitations when working with imbalanced datasets. In scenarios where one class is significantly more frequent than the other (for example, the negative class), a model may achieve high accuracy simply by always predicting the dominant class, without effectively capturing the true structure of the problem.

Definició 17. *Precision measures the proportion of true positives (TP) relative to the total number of positive predictions made by the model, that is, the sum of true positives and false positives (FP). It is defined as:*

$$Precision = \frac{TP}{TP + FP}$$

This metric indicates the degree of confidence in a positive prediction. A model with high precision implies that only a small fraction of the instances classified as positive actually correspond to false positives, meaning that most positive predictions are indeed correct.

Definició 18. *Recall, also called Sensitivity or True Positive Rate (TPR), measures the proportion of true positives (TP) relative to the total number of instances that ically belong to the positive class, that is, the sum of true positives and false negatives (FN). It is expressed by the following formula:*

$$R = \frac{TP}{TP + FN}$$

This metric evaluates the model's ability to correctly identify positive examples. It is particularly useful in scenarios where false negatives entail a significant cost or risk, and where it is crucial to minimize the omission of positive cases.

Definició 19. *The False Positive Rate (FPR), or false positive rate, quantifies the proportion of instances incorrectly classified as positive relative to the total number of examples that actually belong to the negative class. It is defined mathematically as:*

$$FPR = \frac{FP}{FP + TN}$$

This metric evaluates the frequency with which negative examples are incorrectly classified as positive, providing information about the model's false alarm rate.

A.1.6 The ROC Curve

Definició 20. *In a binary classification problem, the model predicts whether an observation belongs to the positive class (1) or the negative class (0). Instead of generating only final predictions (1 or 0), some models (such as logistic regression or neural networks) produce a probability for each prediction. To convert these probabilities into a binary classification, it is necessary to select a threshold (by default, this is usually 0.5). The ROC curve shows how the performance of the model varies as this threshold changes.*

In the ROC curve, the Y-axis represents the *True Positive Rate* (TPR), whereas the X-axis corresponds to the *False Positive Rate* (FPR). Each point on the ROC curve is associated with a different classification threshold τ, which varies over the interval $[0, 1]$. When $\tau = 0$, the model classifies all instances as positive, which maximizes both TPR and FPR, whereas for $\tau = 1$, all instances are classified as negative, resulting in TPR $= 0$ and FPR $= 0$.

In an ideal classification model, the ROC curve would be characterized by an immediate rise to TPR $= 1$ with FPR $= 0$, followed by a horizontal trajectory along the X-axis. In geometric terms, this curve would approach the upper-left vertex of the graph, indicating that the model is capable of correctly identifying all true positives without generating false positives, that is, TPR $= 1$ and FPR $= 0$.

By contrast, if the ROC curve coincides with the diagonal TPR $=$ FPR for all threshold values τ, the model behaves equivalently to random classification, with no discriminative power between classes. In this case, the probability of correctly classifying a positive example is equal to the probability of committing a false positive, which suggests that the model provides no relevant information for classification.

If the ROC curve lies below the diagonal, that is, if for a set of thresholds it holds that TPR $<$ FPR, the model is operating with performance inferior to that of a random classifier. In this situation, the model predicts incorrectly more often than it predicts correctly, indicating an inversion in class assignment and suggesting that a simple inversion of the predictions could improve performance.

A quantitative way of measuring the performance of a model using the ROC curve is the AUC (Area Under the Curve).

Definició 21. *Let there be a binary classification model whose ROC curve describes the relationship between the True Positive Rate and the False Positive Rate as a function of*

a decision threshold τ. The Area Under the ROC Curve (AUC) is defined as the integral of the ROC curve over the interval $[0, 1]$*:*

$$AUC = \int_0^1 TPR(FPR)\, d(FPR)$$

where $TPR(FPR)$ represents the function of the true positive rate as a function of the false positive rate.

The value of AUC provides an aggregate metric of model performance across all classification thresholds. A value of AUC ≈ 1 indicates a model with high discriminative ability between classes, that is, a model that predicts correctly with a high true positive rate and a low false positive rate. By contrast, a value of AUC ≈ 0.5 suggests that the model is no better than random classification, since its TPR is approximately equal to its FPR at all thresholds.

A.2 Model Validation

The purpose of using evaluation metrics in machine learning is to quantify model performance in order to measure, compare, and optimize its behavior, ensuring that it is aligned with the specific objectives of the problem and the needs of the application domain.

However, evaluating a metric on the same dataset used to train the model tends to overestimate its performance due to overfitting. It is therefore necessary to employ strategies that make it possible to estimate model performance on unseen data accurately. Two widely used techniques are considered below.

Train-Test Split

Definició 22. *Train-Test Split is a fundamental technique in machine learning consisting of dividing the dataset D into two disjoint subsets: the training set (D_{train}), used to fit the model parameters, and the test set (D_{test}), used to evaluate performance on unseen data. This separation makes it possible to estimate the model's generalization ability and to prevent overfitting to the training data, ensuring that the evaluation reflects more accurately its performance in real situations.*

A.2.1 K-Fold Cross Validation

Definició 23. *Let $\{(x_i, y_i)\}_{i=1}^n$ be a dataset, where x_i represents the input features and y_i the target variable. The K-Fold Cross Validation technique is a validation method used in machine learning to evaluate the performance of a model more robustly, especially in scenarios with limited data.*

Instead of performing a single split between training and testing, as in *Train-Test Split*, K-fold cross-validation divides D into K disjoint subsets $D_1, D_2, \ldots, D_K$ (called *folds*), such that:

$$D_1 \cup D_2 \cup \cdots \cup D_K = D, \quad D_i \cap D_j = \emptyset, \quad \forall i \neq j.$$

Each subset contains approximately $\frac{n}{K}$ observations. At each iteration k ($k = 1, \ldots, K$), one subset D_k is selected as the validation set, while the remaining $K-1$ subsets constitute the training set:

$$D_{\text{train}}^{(k)} = \bigcup_{\substack{j=1 \\ j \neq k}}^{K} D_j, \quad D_{\text{val}}^{(k)} = D_k.$$

The model $f_\theta^{(k)}$ is trained on $D_{\text{train}}^{(k)}$ and evaluated on $D_{\text{val}}^{(k)}$, obtaining an error given by a *loss function* (or a performance metric). This process is repeated K times, alternating the fold used for validation at each iteration.

Once the K iterations have been completed, the mean of the obtained errors is computed, providing a more stable and reliable estimate of model performance on the entire dataset, reducing the variability of the evaluation and offering a more representative measure of its generalization ability.

A.2.2 Overfitting vs. underfitting

Let $D_{\text{train}} = \{(x_i, y_i)\}_{i=1}^n$ be a training dataset, where $x_i \in X$ represents a feature vector and $y_i \in Y$ is the target variable. The goal is to find a function $f_\phi : X \to Y$, parameterized by ϕ, that minimizes a loss function $L(f_\phi(x), y)$ on the training data and generalizes well to a test set D_{test}.

Definició 24. *A model is said to suffer from underfitting if its representational capacity is insufficient to capture the underlying structure in the data, which results in both high training error and high test error.*

This implies that the model fails to minimize the loss on D_{train}, suggesting that its modeling capacity is too limited. This may occur when the function f_ϕ belongs to a family of functions H of low complexity.

Definició 25. *A model is said to suffer from overfitting when it learns the details of the training set excessively, including noise or spurious patterns, which prevents correct generalization to new data. In this case, the model succeeds in minimizing the error on D_{train}, but its error on D_{test} is significantly larger.*

Overfitting usually occurs when the hypothesis family H is too flexible, allowing f_ϕ to fit each individual sample in D_{train} instead of learning the general structure of the data.

A.3 Introduction to Gradient Boosting Trees

Gradient Boosting is one of the most effective and influential methods within supervised learning, especially when the goal is to obtain highly accurate predictive models. This method combines the power of decision trees as base models with optimization techniques based on gradient descent in order to construct sequential models that progressively improve performance. In this section, we introduce the mathematical foundations involved in the construction of this model.

A.3.1 Decision Tree

A decision tree is a predictive model widely used in machine learning and statistics for decision-making or for making predictions based on data.

Definició 26. *A decision tree is a structure in the form of a directed acyclic graph $T = (V, E)$, where V is the set of nodes, each representing a decision, test, or prediction based on a feature of the dataset, and $E \subseteq V \times V$ is the set of edges representing connections between nodes and establishing hierarchical relationships among them.*

Each node $v \in V$ represents a test or decision based on a feature of the dataset, and each edge $(v_i, v_j) \in E$ represents the outcome of a test performed at node v_i, directing the flow toward node v_j.

A decision tree defines a partition of the input space $\mathbb{R}^d$ into disjoint regions based on decision rules that the tree constructs during training. These decision rules consist of tests or comparisons on the values of the features x_i, with the aim of maximizing the homogeneity of the samples within each region with respect to the response variable y.

At each node of the tree, the algorithm selects a feature X_j and a split threshold s that defines the best way to divide the data into two disjoint subsets:

$$R_1(j, s) = \{X \mid X_j < s\}, \quad R_2(j, s) = \{X \mid X_j \geq s\}$$

The objective of the partition is to find the best combination (X_j, s) that minimizes an impurity function I, which measures the heterogeneity of the data within each subset.

In regression problems, where the target variable y is continuous, node impurity is measured through the Sum of Squared Errors (*Sum of Squared Errors*, SST):

$$SST = \sum_{i=1}^{n} (y_i - \bar{y})^2$$

where y_i is the true value of observation i and $\bar{y}$ is the mean of the values y_i within the node. This metric measures the variability of the data within the node.

In classification problems, the Gini index is a metric that measures the impurity of a node in probabilistic terms. The interpretation of the Gini index is that we are calculating the probability of the event

$$E : \{\text{choosing two elements at random from the node that belong to different classes}\}$$

It is defined as:

$$G = \sum_{k=1}^{K} \hat{p}_k (1 - \hat{p}_k)$$

where $\hat{p}_k$ is the proportion of observations in the node belonging to class k, and K is the total number of classes.

Another metric that also measures the degree of uncertainty or disorder in a node is entropy, defined as:

$$D = -\sum_{k=1}^{K} \hat{p}_k \log(\hat{p}_k)$$

where $\hat{p}_k$ is the proportion of observations belonging to class k, and K is the total number of classes.

In a decision tree, when a node is split into two child nodes, the quality of the split is evaluated through the impurity of the *split*, which is defined as the weighted mean of the impurities of the child nodes. This weighting is carried out by considering the proportion of data in each resulting node.

Let N be the total number of observations in a node before the split, and let N_E and N_D denote the number of observations in the left and right nodes, respectively. We define the weight of each node as

$$w_E = \frac{N_E}{N}, \quad w_D = \frac{N_D}{N}.$$

Let $I(R_E)$ and $I(R_D)$ be the impurity measures in the left and right nodes, respectively. The impurity of the *split* is defined as:

$$I_{\text{split}} = w_E I(R_E) + w_D I(R_D).$$

Once the splitting process stops, each terminal node R_t is associated with a prediction value $\hat{y}_t$ based on the data falling within that region.

In classification problems, the prediction value $\hat{y}_t$ for a terminal node R_t is defined as the mode of the classes in that region.

In regression problems, the prediction value $\hat{y}_t$ for a terminal node R_t is defined as the mean of the target values in that region.

A.3.2 Boosting

Definició 27. *Let $D = \{(X_i, Y_i)\}_{i=1}^{n}$ be a training dataset, where $X_i \in \mathbb{R}^d$ represents a feature vector and Y_i is the target variable. Boosting is a machine learning technique that combines multiple base models $h_m(x)$, called weak learners, in order to construct a final model $F_M(x)$ that is more accurate and robust. In boosting, the models are trained sequentially, so that each subsequent model corrects the errors made by the previous models.*

A.3.3 Gradient Descent Method

In these models, the objective is to find the parameters ϕ that minimize a loss function $L(Y, f_\phi(X))$, which measures the discrepancy between the model predictions $f_\phi(X)$ and the true values.

To solve this problem, the gradient descent method is used, which iteratively adjusts

the parameters in the direction opposite to the gradient of the loss function with respect to ϕ.

Definició 28. *The gradient descent method is an iterative method for finding the minimum of a differentiable function $f : \mathbb{R}^n \to \mathbb{R}$. This minimum may be global or local, depending on the nature of $f(x)$.*

Given a minimization problem:

$$\min_{x \in \mathbb{R}^n} f(x),$$

the gradient descent algorithm follows the following iterative scheme:

- ***Step 1**: Choose a random initial point $x_0 \in \mathbb{R}^n$.*
- ***Step 2**: Compute the gradient of the function at the current point, that is,*

  $$\nabla f(x_t),$$

  which indicates the direction of maximum growth of $f(x)$.
- ***Step 3**: Update the point in the direction opposite to the gradient using the equation:*

  $$x_{t+1} = x_t - \eta \nabla f(x_t),$$

  where:x_t is the current iteration of the point in parameter space. $\eta > 0$ is the learning rate, which controls the step size at each iteration. $\nabla f(x_t)$ is the gradient of f evaluated at x_t.

A.4 Gradient Boosting Trees Method

Definició 29. *Let $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$ be a training dataset. The Gradient Boosting Trees (GBT) method is a supervised learning technique based on the principle of boosting, which seeks to approximate a target function $f_\phi(x)$ by minimizing a loss function $L(y, F(x))$ through a sequential combination of weak models $h_t(x)$, typically decision trees.*

It follows the following iterative scheme:

Initially, an initial model $F_0(x)$ is chosen so as to minimize the loss function $L(y, F(X))$ on the original data:

$$F_0(x) = \arg\min_{\gamma} \sum_{i=1}^{N} L(y_i, \gamma)$$

The initial model $F_0(x)$ has not yet learned any structure. Its only purpose is to find a baseline prediction γ that minimizes the loss on the training data. This initial model $F_0(x)$ serves as the starting point for the iterative Gradient Descent procedure.

At each iteration m:

1. *Computation of Pseudo-Residuals*: For a dataset with N samples, N values r_{im} are computed, which may be interpreted as a vector of partial gradients of size N, $r_m = (r_{1m}, r_{2m}, ..., r_{Nm})$, where

$$r_{im} = -\left[\frac{\partial L(y_i, f(x_i))}{\partial f(x_i)}\right]_{f=F_{m-1}}$$

   This vector represents the direction of maximum increase of the loss function evaluated on the data (x_i, y_i), fixing the model at the previous iteration $F_{m-1}(x)$. These values r_{im} act as target values that a decision tree $h_m(x)$ will attempt to predict.

2. *Fitting the New Model*: A new decision tree $h_m(x)$ is fitted using the pseudo-residuals as labels y_{new}, so that $h_m(x) \approx r_{im}$. This model captures the current errors (pseudo-residuals) as a function of the features x_i.

3. *Model Update*: The prediction is updated by accumulating the model $h_m(x)$ with a learning-rate factor η, so that $F_m(x) = F_{m-1}(x) + \eta h_m(x)$.

Model output: The output of the model is the accumulated prediction $\hat{f}(x) = F_M(x)$. The full expression for $F_M(x)$ after M iterations is

$$F_M(x) = F_0(x) + \eta \sum_{m=1}^{M} h_m(x)$$

B Code

This section presents the various code fragments developed throughout the work, with the aim of analyzing and comparing the performance of different vector representations applied to classification tasks. All code is available in the corresponding *GitHub* repository, which also includes specific execution instructions as well as the scripts used to compare the means of the *AUC-ROC* by means of a hypothesis test. The most relevant scripts are reproduced below so that the reader may consult their contents directly if desired.

https://github.com/tfguabmates25/Codis-treball

C Gibbs Sampling

C.1 Introduction

Although the formulation of the *LDA* model provides an elegant probabilistic description of the generative process of documents, its practical implementation is hindered by the intractability of the exact computation of the posterior distribution of the latent variables, as seen in Section [4.2]. In this setting, Gibbs sampling emerges as an efficient and theoretically well-founded solution. Its appeal lies in the fact that, although the joint distribution is intractable, the full conditional distributions of the hidden topics, as in the case of the assignments z_n in *LDA*, can be obtained in explicit form. This makes it possible to define a Markov chain that, under certain conditions, converges to the target distribution, thereby guaranteeing a coherent exploration of the latent topic space. The operation of this method and its specific application to the computation of thematic representations in the *LDA* model are detailed below, based on (Rebeschini 2018, p. 1).

C.2 Algorithm

Suppose that we wish to obtain samples from a random vector $x = (x_1, \ldots, x_m)$ with a joint distribution $p(x_1, \ldots, x_m)$ that is difficult (or impossible) to compute directly. Nevertheless, we know the conditional distributions (Gelfand 2000, p. 1301):

- $p(x_1 \mid x_2, x_3, \ldots, x_m)$
- $p(x_2 \mid x_1, x_3, \ldots, x_m)$
- etc.

Gibbs sampling is carried out according to the following steps:

1. Initialize each x_i randomly.
2. Iterate the following procedure (for $t = 1, \ldots, T$):
    - Sample $x_1^{(t+1)}$ using $p(x_1 \mid x_2^{(t)}, x_3^{(t)}, \ldots, x_m^{(t)})$.

- Sample $x_2^{(t+1)}$ using $p(x_2 \mid x_1^{(t+1)}, x_3^{(t)}, \ldots, x_m^{(t)})$.
- This process continues until x_m is reached.

We may affirm that Gibbs sampling defines a Markov chain because the state generated at time $t+1$, namely $x^{(t+1)}$, is determined exclusively from the previous state $x^{(t)}$ by using the full conditional distributions. This implies that the evolution of the process does not depend on any state prior to time t, which is precisely the property that characterizes Markov processes.

In the case of interest, we consider a discrete-time Markov chain defined on the finite state space $\mathcal{X} = \mathcal{X}_1 \times \cdots \times \mathcal{X}_d$. In the context of Gibbs sampling, the updates are performed sequentially and are conditionally independent, since each component is updated from the most recent available values of the other coordinates.

Therefore, the transition probability $p_{xx'}$ from the current state $x = (x_1, \ldots, x_d) \in \mathcal{X}$ to a new state $x' = (x'_1, \ldots, x'_d) \in \mathcal{X}$, after one complete Gibbs iteration, is given by the following product:

$$p_{x,x'} = p(x'_1 \mid x_2, \ldots, x_d) \cdot p(x'_2 \mid x'_1, x_3, \ldots, x_d) \cdot \cdots \cdot p(x'_d \mid x'_1, \ldots, x'_{d-1}).$$

C.3 Hammersley–Clifford Theorem and Positivity Condition (Discrete Case)

A fundamental result that theoretically justifies the operation of Gibbs sampling in discrete spaces is the fact that the conditional probabilities completely determine the joint distribution, provided that a certain positivity condition holds. This principle is stated in the well-known *Hammersley–Clifford Theorem*, which we formalize below.

Definició 30. *Let $X_1, \ldots, X_d$ be discrete random variables with finite state spaces $\mathcal{X}_1, \ldots, \mathcal{X}_d$, and let $p(x_1, \ldots, x_d)$ be a probability distribution defined on the product space $\mathcal{X} = \mathcal{X}_1 \times \cdots \times \mathcal{X}_d$. We say that p satisfies the positivity condition if, for every $(x_1, \ldots, x_d) \in \mathcal{X}$,*

$$p_{X_i}(x_i) > 0 \quad \textit{for all } i = 1, \ldots, d \quad \Rightarrow \quad p(x_1, \ldots, x_d) > 0.$$

This is equivalent to stating that the support of p is the Cartesian product of the marginal supports:

$$\mathrm{Supp}(p) = \mathrm{Supp}(p_{X_1}) \times \cdots \times \mathrm{Supp}(p_{X_d}).$$

Teorema 4. *(Hammersley–Clifford, discrete version). Let $p(x_1, \ldots, x_m)$ be a probability distribution on $\mathcal{X}_1 \times \cdots \times \mathcal{X}_m$ satisfying the positivity condition. Then, for any reference point $z = (z_1, \ldots, z_m) \in \mathcal{X}$ such that $p(z) > 0$, one has:*

$$p(x_1, \ldots, x_d) \propto \prod_{j=1}^{d} \frac{p(x_j \mid x_1, \ldots, x_{j-1}, z_{j+1}, \ldots, z_d)}{p(z_j \mid x_1, \ldots, x_{j-1}, z_{j+1}, \ldots, z_d)}.$$

That is, the joint distribution $p(x_1, \ldots, x_d)$ is completely determined (up to a normalization constant) by its full conditionals $p(x_j \mid x_{-j})$. In particular, under the positivity condition, there cannot exist two distinct distributions sharing the same full conditionals.

This theorem provides a formal justification for Gibbs sampling: if we know the full conditional probabilities of a positive joint distribution, then these conditionals uniquely determine the target distribution. Therefore, an algorithm that performs iterative sampling according to these conditionals may converge to the distribution of interest, provided that the associated Markov chain is irreducible and recurrent.

C.4 Invariance of the Limiting Distribution (Discrete Case)

A fundamental aspect in guaranteeing the convergence of Gibbs sampling is that the target distribution $p(x_1, \ldots, x_d)$ be invariant under the transition matrix Π associated with the Markov chain defined by the algorithm. In the discrete case, this fact is verified by showing that:

$$\sum_{x \in \mathcal{X}} p(x)\, p_{(x,x')} = p(x') \quad \text{for all } x' \in \mathcal{X}.$$

The following lemma formalizes this fact:

Lema 5. *Let $p(x_1, \ldots, x_d)$ be a probability distribution on a finite discrete space $\mathcal{X} = \mathcal{X}_1 \times \cdots \times \mathcal{X}_d$, and let $p_{x,x'}$ be the transition probability between two states of the Markov chain generated by Gibbs sampling. Then p is invariant under Π, that is,*

$$\sum_{x \in \mathcal{X}} p(x)\, p_{x,x'} = p(x') \quad \textit{for all } x' \in \mathcal{X}.$$

This result establishes that Gibbs sampling preserves the target distribution p as a stationary distribution. Nevertheless, the fact that p is invariant does not necessarily imply that the chain converges to p from any initial state. In order to ensure asymptotic convergence to the stationary distribution, it is additionally necessary to guarantee that the Markov chain is irreducible and positive recurrent, as will be explained in the following section.

C.5 Irreducibility and Recurrence (Discrete Case)

For Gibbs sampling to converge in practice to the target distribution, it is not sufficient that the latter be invariant under the transition kernel. It is necessary that the associated Markov chain be capable of exploring the entire support of the distribution. This property is guaranteed under the positivity condition through the concepts of irreducibility and recurrence.

Teorema 5. *Let $p(x_1, \ldots, x_d)$ be a probability distribution on the finite space $\mathcal{X} = \mathcal{X}_1 \times \cdots \times \mathcal{X}_d$ satisfying the positivity condition. Then the Markov chain generated by Gibbs sampling is irreducible and positive recurrent.*

From this result, we may directly derive that the chain admits a unique invariant distribution p, which is the limiting distribution of the chain generated by Gibbs sampling through the conditional distributions.